```
!pip install xgboost==1.7.6
```

```
Collecting xgboost==1.7.6
  Downloading xgboost-1.7.6-py3-none-manylinux2014_x86_64.whl.metadata
(1.9 kB)
Requirement already satisfied: numpy in
/home/ec2-user/anaconda3/envs/pytorch/lib/python3.10/site-packages
(from xgboost==1.7.6) (2.2.6)
Requirement already satisfied: scipy in
/home/ec2-user/anaconda3/envs/pytorch/lib/python3.10/site-packages
(from xgboost==1.7.6) (1.15.2)
Downloading xgboost-1.7.6-py3-none-manylinux2014_x86_64.whl (200.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 200.3/200.3 MB 21.0 MB/s
0:00:09m0:00:0100:01
```

```
!pip install scikit-learn==1.4.2
```

```
Collecting scikit-learn==1.4.2
  Downloading scikit_learn-1.4.2-cp310-cp310-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)
Requirement already satisfied: numpy>=1.19.5 in
/home/ec2-user/anaconda3/envs/pytorch/lib/python3.10/site-packages
(from scikit-learn==1.4.2) (2.2.6)
Requirement already satisfied: scipy>=1.6.0 in
/home/ec2-user/anaconda3/envs/pytorch/lib/python3.10/site-packages
(from scikit-learn==1.4.2) (1.15.2)
Requirement already satisfied: joblib>=1.2.0 in
/home/ec2-user/anaconda3/envs/pytorch/lib/python3.10/site-packages
(from scikit-learn==1.4.2) (1.5.3)
Requirement already satisfied: threadpoolctl>=2.0.0 in /home/ec2-
user/anaconda3/envs/pytorch/lib/python3.10/site-packages (from scikit-
learn==1.4.2) (3.6.0)
Downloading scikit_learn-1.4.2-cp310-cp310-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.1 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.1/12.1 MB 100.6 MB/s
0:00:00
pting uninstall: scikit-learn
    Found existing installation: scikit-learn 1.7.2
    Uninstalling scikit-learn-1.7.2:
      Successfully uninstalled scikit-learn-1.7.2
Successfully installed scikit-learn-1.4.2
```

```
import sklearn
import joblib
import pandas as pd
import xgboost

print("scikit-learn:", sklearn.__version__)
print("joblib:", joblib.__version__)
```

```
print("pandas:", pd.__version__)
print("XGBoost:", xgboost.__version__)
```

```
scikit-learn: 1.4.2
joblib: 1.5.3
pandas: 2.3.3
XGBoost: 1.7.6
```

```
# Check that the dataset ZIP is available before starting the
analysis.
from pathlib import Path
import zipfile
dataset_zip_path = Path("/home/ec2-user/SageMaker/dataset.zip")
if not dataset_zip_path.exists():
    raise FileNotFoundError(
        f"Dataset ZIP was not found at {dataset_zip_path}. "
        "Please upload dataset.zip to /content."
    )
print(f"Dataset found: {dataset_zip_path}")
print(f"File size: {dataset_zip_path.stat().st_size / (1024**3):.2f}
GB")
```

```
Dataset found: /home/ec2-user/SageMaker/dataset.zip
File size: 1.00 GB
```

```
from collections import Counter
with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    entries = [entry for entry in zip_file.infolist() if not
entry.is_dir()]
extension_counts = Counter(
    Path(entry.filename).suffix.lower() or "[no extension]"
    for entry in entries
)
top_level_counts = Counter(
    entry.filename.split("/")[0]
    for entry in entries
)
print(f"Total files: {len(entries):,}")
print("\nFiles by extension:")
for extension, count in sorted(extension_counts.items()):
    print(f"  {extension}: {count:,}")
print("\nTop level directories:")
for folder, count in sorted(top_level_counts.items()):
    print(f"  {folder}: {count:,} files")
```

```
Total files: 3,148

Files by extension:
  .csv: 14
  .ipynb: 6
```

```
  .json: 7
  .log: 608
  .md: 1
  .parquet: 2,467
  .png: 19
  .py: 7
  .sample: 13
  .sh: 1
  .txt: 1
  [no extension]: 4

Top level directories:
  .activate.sh: 1 files
  .git: 17 files
  00_raw_datasets: 615 files
  01_aggregated_datasets: 2,476 files
  02_analysis_scripts: 16 files
  03_whole-facility_profiles: 21 files
  README.md: 1 files
  requirements.txt: 1 files
```

```
workload_dirs = sorted({
    entry.filename.split("/")[1]
    for entry in entries
    if entry.filename.startswith("00_raw_datasets/")
    and len(entry.filename.split("/")) >= 3
})
print(f"Raw workload directories: {len(workload_dirs)}")
for workload in workload_dirs:
    print(f" - {workload}")
```

```
Raw workload directories: 5
 - inference_offline_llama3_70b
 - inference_online_finite_llama3_70b
 - inference_online_rate_llama3_70b
 - training_llama2_70b_lora
 - training_stable_diffusion
```

```
with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    readme_files = [
        name for name in zip_file.namelist()
        if name.lower().endswith(".md")
    ]
    if not readme_files:
        raise FileNotFoundError("No README Markdown file was found in
the ZIP archive.")
    readme_name = readme_files[0]
    readme_text = zip_file.read(readme_name).decode("utf-8",
errors="replace")
print(readme_text)
```

GenAI Power Profiles Dataset

This repository contains datasets and analysis scripts related to power consumption profiles of generative AI (GenAI) workloads, including both training and inference workloads.

The first version of the dataset includes the following genAI workloads:

- Training or Fine-Tuning:
    - Llamma2 (LLM)
    - Stable Diffusion (Image Generation)
- Inference (Offline and Online)
    - Llamma3 (LLM)

The goal of this dataset is to support data center research in:

- Infrastructure planning
- Power modeling
- Energy-aware job scheduling
- Load forecasting
- Grid integration studies

<!-- -------------- -->

Citation

If you use this dataset, please cite the associated publication:

> Vercellino et al., “Measurement of Generative AI Workload Power Profiles for Whole-Facility Data Center Infrastructure Planning,” [arXiv:2604.07345](https://arxiv.org/abs/2604.07345) (2026).

The original dataset is available via the NLR Data Catalog: https://data.nlr.gov/submissions/312

<!-- -------------- -->

Repository Structure and Description

*   `00_raw_datasets/`: Contains the original, unmodified datasets as collected using the **WattAMeter** measurement software. Data is provided per individual GPU socket and CPU core. File names with the tag "nvml" refer to GPU, while names with the tag "rapl" refer to CPU. Files may also contain temperature measurements.

*   `01_aggregated_datasets/`: Contains aggregation scripts and aggregated versions of the raw datasets. Typical transformations include summing over GPUs and CPUs, and time resampling to 0.1 or 0.2 seconds. Aggregated datasets are included by default.

```
*   `02_analysis_scripts/`: Contains analysis and plotting scripts used in the accompanying paper for the measured power samples.

* `03_whole-facility_profiles/`: Contains analysis and plotting scripts used in the accompanying paper for the representative whole-facility compute profiles simulated using NLR's Data center Infrastructure Planning, Load Optimization for Energy Efficiency (**DIPLOEE**) modeling tool. Please, refer to the paper for more information on how these synthetic profiles are generated.

<!-- -------------- -->

## Python Environment

Running the scrips in this repository requires a Python environment with a few basic packages. Create and activate and environment using your preferred method (e.g. Python venv, Conda, uv) and install the packages with:

    pip install -r requirements.txt

<!-- -------------- -->

## Example Workflow

All data (raw and aggregated) in the dataset repository, along with the associated analysis plots, has been pre-generated; however, users can run the provided scripts to reproduce and verify the results:

1. Run aggregation scripts (postprocess.py) inside of each folder in `01_aggregated_datasets/`
2. Generate plots and analyze measured data using scripts in `02_analysis_scripts/`
3. Replicate whole-facility plots using scripts in `03_whole-facility_profiles/`


import io
import pandas as pd
with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    parquet_files = sorted(
        name for name in zip_file.namelist()
        if name.lower().endswith(".parquet")
    )
    parquet_counts = Counter(
        path.split("/")[1]
        for path in parquet_files
    )
    print("Parquet files by workload:")
    for workload, count in sorted(parquet_counts.items()):
```

```
        print(f"  {workload}: {count:,}")
    # Select one representative Parquet file from each workload group.
    sample_files = {}
    for path in parquet_files:
        workload = path.split("/")[1]
        sample_files.setdefault(workload, path)
    for workload, path in sorted(sample_files.items()):
        with zip_file.open(path) as file:
            sample_df = pd.read_parquet(io.BytesIO(file.read()))
        print(f"WORKLOAD:{workload}")
        print(f"FILE:{Path(path).name}")
        print(f"Rows:{len(sample_df):,}")
        print(f"Columns:{list(sample_df.columns)}")
        print(sample_df.head(3).to_string())
```

```
Parquet files by workload:
  inference_offline_llama3_70b: 1,200
  inference_online_finite_llama3_70b: 1,026
  inference_online_rate_llama3_70b: 200
  training: 41
WORKLOAD:inference_offline_llama3_70b
FILE:000000.parquet
Rows:511
Columns:['power[W]']
                power[W]
timestep[s]
0.0           792.143065
0.1           991.327360
0.2          1248.529331
WORKLOAD:inference_online_finite_llama3_70b
FILE:000000.parquet
Rows:1,190
Columns:['power[W]']
                power[W]
timestep[s]
0.0           837.754307
0.1           973.837795
0.2          1072.616736
WORKLOAD:inference_online_rate_llama3_70b
FILE:000000.parquet
Rows:179,955
Columns:['power[W]']
                power[W]
timestep[s]
0.000        2066.473043
0.001        2066.474635
0.002        2066.476227
WORKLOAD:training
FILE:000000.parquet
Rows:26,601
```

```
Columns:['power[W]']
                power[W]
timestep[s]
0.0           868.775478
0.2           875.796679
0.4           872.559324
```

```
import json
with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    json_files = sorted(
        name for name in zip_file.namelist()
        if name.lower().endswith(".json")
    )
    print(f"JSON files found: {len(json_files)}")
    for name in json_files:
        print(name)
        try:
            data = json.loads(
                zip_file.read(name).decode("utf-8")
            )
        except json.JSONDecodeError:
            print("Non-standard JSON/JSONL format; skipped for this
inspection.")
            continue
        if isinstance(data, dict) and "experiments" in data:
            experiments = data["experiments"]
            print(f"Experiments: {len(experiments)}")
            if experiments:
                print(json.dumps(experiments[0], indent=2))
        elif isinstance(data, list):
            print(f"Records: {len(data)}")
            if data:
                print(f"Fields: {list(data[0].keys())}")
                print(json.dumps(data[0], indent=2))
        else:
            print(json.dumps(data, indent=2))
```

```
JSON files found: 7
00_raw_datasets/inference_offline_llama3_70b/config.json
{
  "SLURM_CPUS_ON_NODE": "128",
  "SLURM_GPUS_ON_NODE": "4",
  "SLURM_NODELIST": "x3115c0s33b0n0",
  "model": "/projects/aiuserapps/jwillard/hf_models/Llama-3.1-70B-
Instruct"
}
00_raw_datasets/inference_offline_llama3_70b/log.json
Records: 1200
Fields: ['batch_size', 'elapsed', 'end_time', 'max_output_tokens',
'repeat', 'seed', 'start_time']
```

```
{
  "batch_size": 550,
  "elapsed": 51.138686,
  "end_time": "2025-09-18_16:31:03.916427",
  "max_output_tokens": 1024,
  "repeat": 0,
  "seed": 2854,
  "start_time": "2025-09-18_16:30:12.777741"
}
00_raw_datasets/inference_online_finite_llama3_70b/config.json
Experiments: 1026
{
  "_repeat": 0,
  "burstiness": 100000,
  "dataset-name": "hf",
  "dataset-path": "likaixin/InstructCoder",
  "hf-output-len": 256,
  "num-prompts": 1000,
  "request_rate": 10,
  "seed": 0
}
00_raw_datasets/inference_online_finite_llama3_70b/log.json
Records: 1026
Fields: ['_repeat', 'burstiness', 'dataset-name', 'dataset-path',
'end_time', 'hf-output-len', 'id', 'num-prompts', 'request_rate',
'seed', 'start_time']
{
  "_repeat": 0,
  "burstiness": 100000,
  "dataset-name": "hf",
  "dataset-path": "likaixin/InstructCoder",
  "end_time": "2025-11-30_15:42:02.015074",
  "hf-output-len": 256,
  "id": "000001",
  "num-prompts": 1000,
  "request_rate": 10,
  "seed": 0,
  "start_time": "2025-11-30_15:39:30.303018"
}
00_raw_datasets/inference_online_finite_llama3_70b/log_vllm.json
Non-standard JSON/JSONL format; skipped for this inspection.
00_raw_datasets/inference_online_rate_llama3_70b/config.json
Experiments: 200
{
  "burstiness": 100000,
  "dataset-name": "hf",
  "dataset-path": "likaixin/InstructCoder",
  "num-prompts": 300,
  "request_rate": 1
```

```
}
00_raw_datasets/inference_online_rate_llama3_70b/log.json
Records: 200
Fields: ['burstiness', 'dataset-name', 'dataset-path', 'end_time',
'id', 'num-prompts', 'request_rate', 'start_time']
{
  "burstiness": 100000,
  "dataset-name": "hf",
  "dataset-path": "likaixin/InstructCoder",
  "end_time": "2026-01-05_05:35:52.751778",
  "id": "000001",
  "num-prompts": 300,
  "request_rate": 1,
  "start_time": "2026-01-05_05:30:32.714319"
}

with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    mapping_checks = {
        "inference_offline_llama3_70b":
            "00_raw_datasets/inference_offline_llama3_70b/log.json",

        "inference_online_finite_llama3_70b":

"00_raw_datasets/inference_online_finite_llama3_70b/log.json",

        "inference_online_rate_llama3_70b":

"00_raw_datasets/inference_online_rate_llama3_70b/log.json",
    }
    for workload, log_path in mapping_checks.items():
        log_records = json.loads(
            zip_file.read(log_path).decode("utf-8")
        )
        parquet_count = sum(
            1
            for path in parquet_files
            if f"01_aggregated_datasets/{workload}/results/" in path
        )
        status = "OK" if len(log_records) == parquet_count else
"MISMATCH"
        print(
            f"{workload}: "
            f"metadata records={len(log_records):,}, "
            f"Parquet files={parquet_count:,} -> {status}"
        )

inference_offline_llama3_70b: metadata records=1,200, Parquet
files=1,200 -> OK
inference_online_finite_llama3_70b: metadata records=1,026, Parquet
files=1,026 -> OK
```

```
inference_online_rate_llama3_70b: metadata records=200, Parquet
files=200 -> OK
```

```
metadata_paths = {
    "offline":
"01_aggregated_datasets/inference_offline_llama3_70b/metadata.csv",
    "online_finite":
"01_aggregated_datasets/inference_online_finite_llama3_70b/metadata.cs
v",
    "online_rate":
"01_aggregated_datasets/inference_online_rate_llama3_70b/metadata.csv"
,
    "training": "01_aggregated_datasets/training/metadata.csv",
}
metadata = {}
with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    for workload, path in metadata_paths.items():
        with zip_file.open(path) as file:
            metadata[workload] = pd.read_csv(file)
        print(f"{workload}:{metadata[workload].shape}")
        print(f"Columns:{list(metadata[workload].columns)}")
        display(metadata[workload].head(3))
```

```
offline:(1200, 10)
Columns:['batch_size', 'elapsed', 'end_time', 'max_output_tokens',
'repeat', 'seed', 'start_time', 'path_run', 'peak_power[W]',
'mean_power[W]']
```

```
   batch_size    elapsed                    end_time
max_output_tokens  \
0         550  51.138686  2025-09-18 16:31:03.916427
1024
1         550  47.384165  2025-09-18 16:31:53.300838
1024
2         550  48.965850  2025-09-18 16:32:44.266972
1024

   repeat  seed                   start_time  \
0       0  2854  2025-09-18 16:30:12.777741
1       1  2854  2025-09-18 16:31:05.916673
2       2  2854  2025-09-18 16:31:55.301122

                                            path_run  peak_power[W]  \
0  /projects/aiuserapps/genai_power_dataset/01_ag...    3053.759065
1  /projects/aiuserapps/genai_power_dataset/01_ag...    3048.839824
2  /projects/aiuserapps/genai_power_dataset/01_ag...    3042.525523

   mean_power[W]
0    2822.104293
```

```
1    2814.350863
2    2798.014121

online_finite:(1026, 72)
Columns:['_repeat', 'burstiness_x', 'dataset-name', 'dataset-path',
'end_time_exc', 'hf-output-len', 'id', 'num-prompts',
'request_rate_x', 'seed', 'start_time_exc', 'date', 'endpoint_type',
'backend', 'label', 'model_id', 'tokenizer_id', 'num_prompts',
'request_rate_y', 'burstiness_y', 'max_concurrency', 'duration',
'completed', 'total_input_tokens', 'total_output_tokens',
'request_throughput', 'request_goodput', 'output_throughput',
'total_token_throughput', 'max_output_tokens_per_s',
'max_concurrent_requests', 'mean_ttft_ms', 'median_ttft_ms',
'std_ttft_ms', 'p10_ttft_ms', 'p30_ttft_ms', 'p50_ttft_ms',
'p70_ttft_ms', 'p90_ttft_ms', 'mean_tpot_ms', 'median_tpot_ms',
'std_tpot_ms', 'p10_tpot_ms', 'p30_tpot_ms', 'p50_tpot_ms',
'p70_tpot_ms', 'p90_tpot_ms', 'mean_itl_ms', 'median_itl_ms',
'std_itl_ms', 'p10_itl_ms', 'p30_itl_ms', 'p50_itl_ms', 'p70_itl_ms',
'p90_itl_ms', 'mean_e2el_ms', 'median_e2el_ms', 'std_e2el_ms',
'p10_e2el_ms', 'p30_e2el_ms', 'p50_e2el_ms', 'p70_e2el_ms',
'p90_e2el_ms', 'start_time_vllm', 'end_time_vllm', 'start_time',
'end_time', 'run_completed', 'execution_time_seconds', 'path_run',
'peak_power[W]', 'mean_power[W]']

   _repeat  burstiness_x dataset-name            dataset-path  \
0        0        100000           hf  likaixin/InstructCoder
1        0        100000           hf  likaixin/InstructCoder
2        0        100000           hf  likaixin/InstructCoder

                  end_time_exc  hf-output-len  id  num-prompts
request_rate_x  \
0  2025-11-30 15:41:52.015074            256   1         1000
10
1  2025-11-30 15:43:18.318080            256   2         1000
20
2  2025-11-30 15:44:27.872271            256   3         1000
30

   seed  ...  p90_e2el_ms      start_time_vllm
end_time_vllm  \
0     0  ...  5114.628401  2025-11-30 15:41:38  2025-11-30
15:43:22.123269388
1     0  ...  5457.068536  2025-11-30 15:43:18  2025-11-30
15:44:12.522184050
2     0  ...  5892.459001  2025-11-30 15:44:27  2025-11-30
15:45:05.159787518

                    start_time                    end_time
run_completed  \
0  2025-11-30 15:39:40.303018  2025-11-30 15:41:52.015074
```

```
True
1  2025-11-30 15:42:12.017302  2025-11-30 15:43:18.318080
True
2  2025-11-30 15:43:38.320088  2025-11-30 15:44:27.872271
True

  execution_time_seconds
path_run  \
0                  118.9
/projects/aiuserapps/genai_power_dataset/01_ag...
1                   56.0
/projects/aiuserapps/genai_power_dataset/01_ag...
2                   39.6
/projects/aiuserapps/genai_power_dataset/01_ag...

   peak_power[W]  mean_power[W]
0    2351.683114    2085.306406
1    2348.551505    2265.948935
2    2601.319313    2446.739765

[3 rows x 72 columns]

online_rate:(200, 15)
Columns:['burstiness', 'dataset-name', 'dataset-path', 'end_time_exc',
'id', 'num-prompts', 'request_rate', 'start_time_exc', 'start_time',
'end_time', 'run_completed', 'execution_time_seconds', 'path_run',
'peak_power[W]', 'mean_power[W]']

   burstiness dataset-name           dataset-path  \
0      100000           hf  likaixin/InstructCoder
1      100000           hf  likaixin/InstructCoder
2      100000           hf  likaixin/InstructCoder

                  end_time_exc  id  num-prompts  request_rate  \
0  2026-01-05 05:34:32.714319   1          300             1
1  2026-01-05 05:39:52.753836   2          600             2
2  2026-01-05 05:45:12.801867   3          900             3

                start_time_exc                  start_time  \
0  2026-01-05 05:31:32.714319  2026-01-05 05:31:32.714319
1  2026-01-05 05:36:52.753836  2026-01-05 05:36:52.753836
2  2026-01-05 05:42:12.801867  2026-01-05 05:42:12.801867

                     end_time  run_completed
execution_time_seconds  \
0  2026-01-05 05:34:32.714319           True                  179.954

1  2026-01-05 05:39:52.753836           True                  179.846

2  2026-01-05 05:45:12.801867           True                  179.943
```

```
                                                path_run  peak_power[W]  \
0  /projects/aiuserapps/genai_power_dataset/01_ag...    2097.831345
1  /projects/aiuserapps/genai_power_dataset/01_ag...    2118.523541
2  /projects/aiuserapps/genai_power_dataset/01_ag...    2140.690436

   mean_power[W]
0    2069.548985
1    2085.325478
2    2088.905829

training:(41, 6)
Columns:['Unnamed: 0', 'model', 'nodes', 'repeat', 'path_save',
'slurmid']

   Unnamed: 0            model  nodes  repeat  \
0           0  llama2_70b_lora      2       0
1           1  llama2_70b_lora      2       1
2           2  llama2_70b_lora      2       2

                        path_save   slurmid
0  training/results/000000.parquet  10742795
1  training/results/000001.parquet  10742796
2  training/results/000002.parquet  10742797
```

```python
training_power_rows = []
with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    training_parquet_files = sorted(
        name
        for name in zip_file.namelist()
        if name.startswith("01_aggregated_datasets/training/results/")
        and name.lower().endswith(".parquet")
    )
    for metadata_index, path in enumerate(training_parquet_files):
        with zip_file.open(path) as file:
            power_values = pd.read_parquet(
                io.BytesIO(file.read())
            )["power[W]"]
        training_power_rows.append({
            "metadata_index": metadata_index,
            "mean_power[W]": power_values.mean(),
            "peak_power[W]": power_values.max(),
            "power_samples": len(power_values),
        })
training_power = pd.DataFrame(training_power_rows)
training_metadata = metadata["training"].reset_index(drop=True).copy()
training_metadata["metadata_index"] = training_metadata.index
training_full = (
    training_metadata
    .merge(training_power, on="metadata_index", how="left")
```

```
    .drop(columns="metadata_index")
)
assert training_full[
    ["mean_power[W]", "peak_power[W]"]
].notna().all().all()
print(
    f"Training executions: {len(training_full):,} "
    "| Missing power summaries: "
    f"{training_full[['mean_power[W]',
'peak_power[W]']].isna().sum().sum()}"
)
display(training_full.head())
```

```
Training executions: 41 | Missing power summaries: 0
```

```
   Unnamed: 0            model  nodes  repeat  \
0           0  llama2_70b_lora      2       0
1           1  llama2_70b_lora      2       1
2           2  llama2_70b_lora      2       2
3           3  llama2_70b_lora      2       3
4           4  llama2_70b_lora      2       4

                        path_save   slurmid  mean_power[W]
peak_power[W]  \
0  training/results/000000.parquet  10742795    5267.813212
6144.482505
1  training/results/000001.parquet  10742796    5284.503512
6124.141955
2  training/results/000002.parquet  10742797    5278.125021
6115.386270
3  training/results/000003.parquet  10742798    5226.846200
6115.803822
4  training/results/000004.parquet  10742800    5259.117817
6143.958648

   power_samples
0          26601
1          29169
2          26562
3          26595
4          26567
```

```
raw_sample_paths = [
    "00_raw_datasets/inference_offline_llama3_70b/"
    "nvml_wattameter_x3115c0s33b0n0.log",
    "00_raw_datasets/inference_offline_llama3_70b/"
    "rapl_wattameter_x3115c0s33b0n0.log",
]
with zipfile.ZipFile(dataset_zip_path, "r") as zip_file:
    for path in raw_sample_paths:
```

```
        print(path)
        with zip_file.open(path) as file:
            for _ in range(3):
                line = file.readline().decode(
                    "utf-8",
                    errors="replace"
                ).strip()
                print(line[:160])
```

```
00_raw_datasets/inference_offline_llama3_70b/
nvml_wattameter_x3115c0s33b0n0.log
# 2025-09-18_16:22:44.314867 - Power data for run 10720618
# 2025-09-18_16:22:44.314867 - Tracking started
# timestamp                  reading-time[ns] gpu-0[mW] gpu-1[mW] gpu-
2[mW] gpu-3[mW] gpu-0[C] gpu-1[C] gpu-2[C] gpu-3[C]
00_raw_datasets/inference_offline_llama3_70b/rapl_wattameter_x3115c0s3
3b0n0.log
# 2025-09-18_16:22:44.314867 - Power data for run 10720618
# 2025-09-18_16:22:44.314867 - Tracking started
# timestamp                  reading-time[ns] cpu-0[uJ] cpu-0-core[uJ]
cpu-1[uJ] cpu-1-core[uJ] cpu-0[W] cpu-0-core[W] cpu-1[W] cpu-1-core[W]
```

```
excluded_fields = {
    "mean_power[W]",
    "peak_power[W]",
    "elapsed",
    "execution_time_seconds",
    "path_run",
    "path_save",
    "slurmid",
    "id",
    "Unnamed: 0",
    "start_time",
    "end_time",
    "start_time_exc",
    "end_time_exc",
    "start_time_vllm",
    "end_time_vllm",
}
for workload in ["offline", "online_finite", "online_rate",
"training"]:
    workload_df = (
        training_full
        if workload == "training"
        else metadata[workload]
    )
    varying_fields = [
        (column, workload_df[column].nunique(dropna=True))
        for column in workload_df.columns
        if column not in excluded_fields
```

```
        and workload_df[column].nunique(dropna=True) > 1
    ]
    print(f"{workload}: {len(workload_df):,} executions")
    if varying_fields:
        print("Varying configuration fields:")
        for column, unique_values in varying_fields:
            print(f"  {column}: {unique_values} unique values")
    else:
        print("No varying configuration fields found.")
```

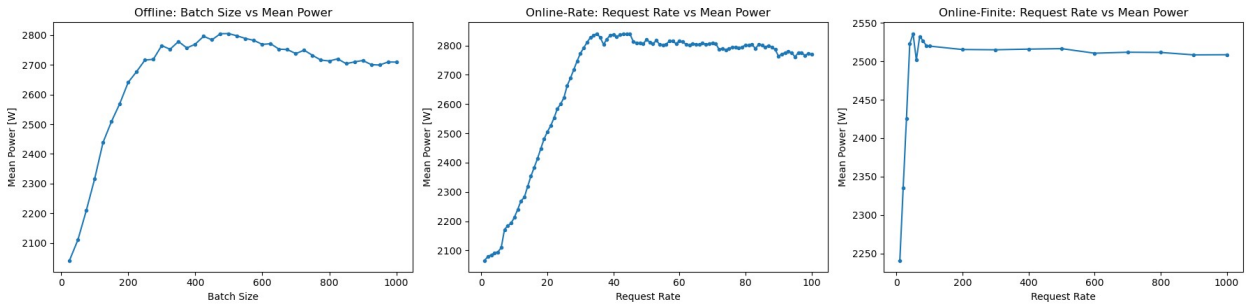
```
offline: 1,200 executions
Varying configuration fields:
  batch_size: 40 unique values
  max_output_tokens: 2 unique values
  repeat: 3 unique values
  seed: 400 unique values
online_finite: 1,026 executions
Varying configuration fields:
  _repeat: 3 unique values
  dataset-path: 2 unique values
  hf-output-len: 3 unique values
  request_rate_x: 19 unique values
  seed: 3 unique values
  date: 1024 unique values
  request_rate_y: 19 unique values
  duration: 1024 unique values
  total_input_tokens: 6 unique values
  total_output_tokens: 993 unique values
  request_throughput: 1024 unique values
  output_throughput: 1024 unique values
  total_token_throughput: 1024 unique values
  max_output_tokens_per_s: 810 unique values
  max_concurrent_requests: 356 unique values
  mean_ttft_ms: 1024 unique values
  median_ttft_ms: 1024 unique values
  std_ttft_ms: 1024 unique values
  p10_ttft_ms: 1024 unique values
  p30_ttft_ms: 1024 unique values
  p50_ttft_ms: 1024 unique values
  p70_ttft_ms: 1024 unique values
  p90_ttft_ms: 1024 unique values
  mean_tpot_ms: 1024 unique values
  median_tpot_ms: 1024 unique values
  std_tpot_ms: 1024 unique values
  p10_tpot_ms: 1024 unique values
  p30_tpot_ms: 1024 unique values
  p50_tpot_ms: 1024 unique values
  p70_tpot_ms: 1024 unique values
  p90_tpot_ms: 1024 unique values
  mean_itl_ms: 1024 unique values
```

```
  median_itl_ms: 1024 unique values
  std_itl_ms: 1024 unique values
  p10_itl_ms: 1024 unique values
  p30_itl_ms: 1024 unique values
  p50_itl_ms: 1024 unique values
  p70_itl_ms: 1024 unique values
  p90_itl_ms: 1024 unique values
  mean_e2el_ms: 1024 unique values
  median_e2el_ms: 1024 unique values
  std_e2el_ms: 1024 unique values
  p10_e2el_ms: 1024 unique values
  p30_e2el_ms: 1024 unique values
  p50_e2el_ms: 1024 unique values
  p70_e2el_ms: 1024 unique values
  p90_e2el_ms: 1024 unique values
  run_completed: 2 unique values
online_rate: 200 executions
Varying configuration fields:
  dataset-path: 2 unique values
  num-prompts: 100 unique values
  request_rate: 100 unique values
training: 41 executions
Varying configuration fields:
  model: 2 unique values
  nodes: 4 unique values
  repeat: 6 unique values
  power_samples: 40 unique values
```

```
import matplotlib.pyplot as plt
offline_power = (
    metadata["offline"]
    .groupby("batch_size")["mean_power[W]"]
    .mean()
)
online_rate_power = (
    metadata["online_rate"]
    .groupby("request_rate")["mean_power[W]"]
    .mean()
)
online_finite_power = (
    metadata["online_finite"]
    .groupby("request_rate_x")["mean_power[W]"]
    .mean()
)
fig, axes = plt.subplots(1, 3, figsize=(18, 4.5))
axes[0].plot(
    offline_power.index,
    offline_power.values,
    marker="o",
    markersize=3
```

```
)
axes[0].set_title("Offline: Batch Size vs Mean Power")
axes[0].set_xlabel("Batch Size")
axes[0].set_ylabel("Mean Power [W]")
axes[1].plot(
    online_rate_power.index,
    online_rate_power.values,
    marker="o",
    markersize=3
)
axes[1].set_title("Online-Rate: Request Rate vs Mean Power")
axes[1].set_xlabel("Request Rate")
axes[1].set_ylabel("Mean Power [W]")
axes[2].plot(
    online_finite_power.index,
    online_finite_power.values,
    marker="o",
    markersize=3
)
axes[2].set_title("Online-Finite: Request Rate vs Mean Power")
axes[2].set_xlabel("Request Rate")
axes[2].set_ylabel("Mean Power [W]")
plt.tight_layout()
plt.show()
```

```
Matplotlib is building the font cache; this may take a moment.
```




```
online_finite_df = metadata["online_finite"]
low_power_runs = online_finite_df[
    online_finite_df["mean_power[W]"] < 1000
]
display_columns = [
    column
    for column in [
        "id",
        "request_rate_x",
        "hf-output-len",
        "dataset-path",
        "duration",
```

```
        "completed",
        "run_completed",
        "mean_power[W]",
    ]
    if column in low_power_runs.columns
]
print(f"Low-power executions identified: {len(low_power_runs)}")
display(low_power_runs[display_columns])
print("\nCompletion status where available:")
for workload in [
    "offline",
    "online_finite",
    "online_rate",
    "training",
]:
    workload_df = (
        training_full
        if workload == "training"
        else metadata[workload]
    )
    if "run_completed" in workload_df.columns:
        print(
            f"  {workload}: "
            f"{workload_df['run_completed'].value_counts().to_dict()}"
        )
    else:
        print(
            f"  {workload}: run_completed field not available "
            f"({len(workload_df):,} executions)"
        )
```

```
Low-power executions identified: 2
```

	id	request_rate_x	hf-output-len	dataset-path
686	687	30	256	likaixin/InstructCoder
708	709	60	256	mgoin/mlperf-inference-llama2-data

	duration	completed	run_completed	mean_power[W]
686	NaN	NaN	False	743.649697
708	NaN	NaN	False	739.054963

```
Completion status where available:
  offline: run_completed field not available (1,200 executions)
  online_finite: {True: 1024, False: 2}
  online_rate: {True: 200}
  training: run_completed field not available (41 executions)
```

```
# Create a clean execution level dataset for the preprocessing
notebook.
# Only valid inference executions and modeling variables are retained.
offline = metadata["offline"].copy()
offline["workload_type"] = "offline_inference"
offline_modeling = offline.rename(
    columns={
        "max_output_tokens": "max_output_tokens",
        "mean_power[W]": "mean_power_w",
    }
)[
    [
        "workload_type",
        "batch_size",
        "max_output_tokens",
        "mean_power_w",
    ]
]
online_finite = metadata["online_finite"].copy()
# Remove the two incomplete executions identified during EDA.
online_finite = online_finite[
    online_finite["run_completed"] == True
].copy()
online_finite["workload_type"] = "online_finite_inference"
online_finite_modeling = online_finite.rename(
    columns={
        "request_rate_x": "request_rate",
        "hf-output-len": "output_length",
        "dataset-path": "dataset",
        "mean_power[W]": "mean_power_w",
    }
)[
    [
        "workload_type",
        "request_rate",
        "output_length",
        "dataset",
        "mean_power_w",
    ]
]
online_rate = metadata["online_rate"].copy()
online_rate["workload_type"] = "online_rate_inference"
online_rate_modeling = online_rate.rename(
    columns={
        "request_rate": "request_rate",
        "dataset-path": "dataset",
        "mean_power[W]": "mean_power_w",
    }
)[
    [
```

```
        "workload_type",
        "request_rate",
        "dataset",
        "mean_power_w",
    ]
]
# Combine the three inference workload types.
modeling_dataset = pd.concat(
    [
        offline_modeling,
        online_finite_modeling,
        online_rate_modeling,
    ],
    ignore_index=True
)
print(f"Final modeling dataset shape: {modeling_dataset.shape}")
print("\nWorkload counts:")
print(modeling_dataset["workload_type"].value_counts())
print("\nMissing values:")
print(modeling_dataset.isna().sum())
display(modeling_dataset.head())
```

```
Final modeling dataset shape: (2424, 7)

Workload counts:
workload_type
offline_inference          1200
online_finite_inference    1024
online_rate_inference       200
Name: count, dtype: int64

Missing values:
workload_type           0
batch_size           1224
max_output_tokens    1224
mean_power_w            0
request_rate         1200
output_length        1400
dataset              1200
dtype: int64
```

	workload_type	batch_size	max_output_tokens	mean_power_w	request_rate	output_length	dataset
0	offline_inference	550.0	1024.0	2822.104293	NaN	NaN	NaN
1	offline_inference	550.0	1024.0	2814.350863			
2	offline_inference	550.0	1024.0	2798.014121			
3	offline_inference	275.0	1024.0	2694.764568			
4	offline_inference	275.0	1024.0	2696.992151			

```
1            NaN             NaN     NaN
2            NaN             NaN     NaN
3            NaN             NaN     NaN
4            NaN             NaN     NaN
```

```
from pathlib import Path
dataset_processed_dir = Path(
    "/home/ec2-user/SageMaker/dataset-processed"
)
dataset_processed_dir.mkdir(
    parents=True,
    exist_ok=True
)
modeling_dataset_path = (
    dataset_processed_dir / "inference_modeling_dataset.csv"
)
modeling_dataset.to_csv(
    modeling_dataset_path,
    index=False
)
print(f"Saved modeling dataset to: {modeling_dataset_path}")
print(f"Rows: {len(modeling_dataset):,}")
print(f"Columns: {len(modeling_dataset.columns)}")
```

```
Saved modeling dataset to: /home/ec2-user/SageMaker/dataset-
processed/inference_modeling_dataset.csv
Rows: 2,424
Columns: 7
```

```
import pandas as pd
feature_inventory = pd.DataFrame([
    {"field": "workload_type", "role": "Predictor", "reason":
"Distinguishes workload behaviors."},
    {"field": "batch_size", "role": "Predictor", "reason": "Offline
inference only; known before execution."},
    {"field": "max_output_tokens", "role": "Predictor", "reason":
"Offline inference only; known before execution."},
    {"field": "request_rate", "role": "Predictor", "reason": "Online
inference only; configured rate."},
    {"field": "output_length", "role": "Predictor", "reason": "Online-
finite only; configured output length."},
    {"field": "dataset", "role": "Predictor", "reason": "Input dataset
configuration (online)."},
    {"field": "num_prompts", "role": "Excluded", "reason": "Constant
for online-finite; perfectly collinear with request_rate (corr=1.0)
for online-rate."},
    {"field": "mean_power[W]", "role": "Target", "reason": "Execution-
level mean power."},
    {"field": "peak_power[W]", "role": "Excluded", "reason": "Same-
execution power outcome → target leakage."},
```

```
    {"field": "elapsed / execution_time_seconds", "role": "Excluded",
"reason": "Measured during/after execution."},
    {"field": "throughput / latency metrics", "role": "Excluded",
"reason": "Post-execution performance outcomes."},
    {"field": "completed / run_completed", "role": "Filtering only",
"reason": "Used to drop incomplete runs, never a predictor."},
    {"field": "id / seed / repeat / path / timestamps / slurmid",
"role": "Excluded", "reason": "Identifiers, not workload
characteristics."},
])
display(feature_inventory)
```

```
                                                 field
role  \
0                                       workload_type       Predictor

1                                          batch_size       Predictor

2                                  max_output_tokens       Predictor

3                                       request_rate       Predictor

4                                       output_length       Predictor

5                                             dataset       Predictor

6                                         num_prompts        Excluded

7                                       mean_power[W]          Target

8                                       peak_power[W]        Excluded

9                     elapsed / execution_time_seconds        Excluded

10                        throughput / latency metrics        Excluded

11                           completed / run_completed  Filtering only

12  id / seed / repeat / path / timestamps / slurmid        Excluded


                                                 reason
0                     Distinguishes workload behaviors.
1       Offline inference only; known before execution.
2       Offline inference only; known before execution.
3               Online inference only; configured rate.
4         Online-finite only; configured output length.
5                 Input dataset configuration (online).
6   Constant for online-finite; perfectly collinea...
7                           Execution-level mean power.
8        Same-execution power outcome → target leakage.
```

```
9                    Measured during/after execution.
10               Post-execution performance outcomes.
11   Used to drop incomplete runs, never a predictor.
12         Identifiers, not workload characteristics.
```

```
from pathlib import Path
import pandas as pd
# Load the execution level modeling dataset created by 01_EDA.ipynb.
modeling_dataset_path = Path(

"/home/ec2-user/SageMaker/dataset-processed/inference_modeling_dataset
.csv"
)
if not modeling_dataset_path.exists():
    raise FileNotFoundError(
        f"Modeling dataset not found at: {modeling_dataset_path}"
    )
inference_dataset = pd.read_csv(modeling_dataset_path)
print(f"Dataset shape: {inference_dataset.shape}")
print("\nWorkload counts:")
print(inference_dataset["workload_type"].value_counts().to_string())
print("\nMissing values:")
print(inference_dataset.isna().sum().to_string())
display(inference_dataset.head())
```

```
Dataset shape: (2424, 7)

Workload counts:
workload_type
offline_inference          1200
online_finite_inference    1024
online_rate_inference       200

Missing values:
workload_type           0
batch_size           1224
max_output_tokens    1224
mean_power_w            0
request_rate         1200
output_length        1400
dataset              1200
```

```
       workload_type  batch_size  max_output_tokens  mean_power_w  \
0  offline_inference       550.0             1024.0   2822.104293
1  offline_inference       550.0             1024.0   2814.350863
2  offline_inference       550.0             1024.0   2798.014121
3  offline_inference       275.0             1024.0   2694.764568
4  offline_inference       275.0             1024.0   2696.992151

   request_rate  output_length dataset
```

```
0            NaN             NaN     NaN
1            NaN             NaN     NaN
2            NaN             NaN     NaN
3            NaN             NaN     NaN
4            NaN             NaN     NaN
```

```
feature_columns = ["workload_type", "batch_size", "max_output_tokens",
                   "request_rate", "output_length", "dataset"]
target_column = "mean_power_w"
leakage_fields = {
    "mean_power[W]": "target", "peak_power[W]": "same-execution power
outcome",
    "elapsed": "execution outcome", "execution_time_seconds":
"measured duration",
    "completed": "execution status", "run_completed": "execution
status",
    "id": "identifier", "seed": "identifier", "repeat": "identifier",
    "path_run": "identifier", "slurmid": "identifier",
}
leak_in_features = [c for c in leakage_fields if c in feature_columns]
print("Leakage fields in predictors:", leak_in_features or "NONE")
print("Target in predictors:", target_column in feature_columns)
final_dataset = inference_dataset[feature_columns +
[target_column]].copy()
assert final_dataset[target_column].notna().all()
print(f"\nFinal dataset: {final_dataset.shape}, target missing:
{final_dataset[target_column].isna().sum()}")
```

```
Leakage fields in predictors: NONE
Target in predictors: False

Final dataset: (2424, 7), target missing: 0
```

```
from sklearn.model_selection import train_test_split
X = final_dataset[feature_columns].copy()
y = final_dataset[target_column].copy()
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42,
stratify=X["workload_type"])
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42,
stratify=X_temp["workload_type"])
for name, part in [("train", X_train), ("validation", X_val), ("test",
X_test)]:
    print(f"{name}: {len(part)} rows |
{part['workload_type'].value_counts().to_dict()}")
# No overlap between partitions
assert not (set(X_train.index) & set(X_val.index))
assert not (set(X_train.index) & set(X_test.index))
```

```
assert not (set(X_val.index) & set(X_test.index))
print("Partition overlap: 0 rows everywhere")
```

```
train: 1696 rows | {'offline_inference': 840,
'online_finite_inference': 716, 'online_rate_inference': 140}
validation: 364 rows | {'offline_inference': 180,
'online_finite_inference': 154, 'online_rate_inference': 30}
test: 364 rows | {'offline_inference': 180, 'online_finite_inference':
154, 'online_rate_inference': 30}
Partition overlap: 0 rows everywhere
```

```
import numpy as np
import joblib
from pathlib import Path
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
# Define the persistent output directory.
processed_dir = Path(
    "/home/ec2-user/SageMaker/dataset-modeling"
)
processed_dir.mkdir(
    parents=True,
    exist_ok=True
)
# Define feature groups.
categorical_features = [
    "workload_type",
    "dataset",
]
numerical_features = [
    "batch_size",
    "max_output_tokens",
    "request_rate",
    "output_length",
]
# Build the numerical preprocessing pipeline.
numerical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="median",
            add_indicator=True,
        ),
    ),
    (
        "scaler",
        StandardScaler(),
    ),
```

```
])
# Build the categorical preprocessing pipeline.
categorical_pipeline = Pipeline([
    (
        "imputer",
        SimpleImputer(
            strategy="constant",
            fill_value="not_applicable",
        ),
    ),
    (
        "encoder",
        OneHotEncoder(
            handle_unknown="ignore",
            sparse_output=False,
        ),
    ),
])
# Combine numerical and categorical preprocessing.
preprocessor = ColumnTransformer([
    (
        "numerical",
        numerical_pipeline,
        numerical_features,
    ),
    (
        "categorical",
        categorical_pipeline,
        categorical_features,
    ),
])
# Fit the preprocessing pipeline on training data ONLY.
preprocessor.fit(X_train)
# Transform all partitions using the fitted training only pipeline.
X_train_p = preprocessor.transform(X_train)
X_val_p = preprocessor.transform(X_val)
X_test_p = preprocessor.transform(X_test)
# Retrieve the generated feature names.
feature_names = preprocessor.get_feature_names_out()
# Validate that preprocessing produced finite values.
for name, arr in [
    ("train", X_train_p),
    ("validation", X_val_p),
    ("test", X_test_p),
]:
    assert np.isfinite(arr).all(), (
        f"Non-finite values detected in processed {name} data."
    )
# Save the fitted preprocessing pipeline.
```

```
preprocessor_path = processed_dir / "preprocessor.joblib"
joblib.dump(
    preprocessor,
    preprocessor_path,
)
print(
    f"Processed shapes: "
    f"train {X_train_p.shape}, "
    f"validation {X_val_p.shape}, "
    f"test {X_test_p.shape}"
)
print(f"\nNumber of processed features:{len(feature_names)}")
print("\nProcessed features:")
for feature in feature_names:
    print(f"-{feature}")
print(f"\nFitted preprocessor saved to:")
print(preprocessor_path)
```

```
Processed shapes: train (1696, 14), validation (364, 14), test (364,
14)

Number of processed features:14

Processed features:
-numerical__batch_size
-numerical__max_output_tokens
-numerical__request_rate
-numerical__output_length
-numerical__missingindicator_batch_size
-numerical__missingindicator_max_output_tokens
-numerical__missingindicator_request_rate
-numerical__missingindicator_output_length
-categorical__workload_type_offline_inference
-categorical__workload_type_online_finite_inference
-categorical__workload_type_online_rate_inference
-categorical__dataset_likaixin/InstructCoder
-categorical__dataset_mgoin/mlperf-inference-llama2-data
-categorical__dataset_not_applicable

Fitted preprocessor saved to:
/home/ec2-user/SageMaker/dataset-modeling/preprocessor.joblib
```

```
from pathlib import Path
import os
# Save processed datasets to Google Drive for persistence across Colab
sessions.
processed_dir = Path(
    "/home/ec2-user/SageMaker/dataset-modeling"
)
processed_dir.mkdir(parents=True, exist_ok=True)
```

```
# Features as DataFrames with proper column names; targets as one
column frames.
pd.DataFrame(
    X_train_p, columns=feature_names
).to_parquet(processed_dir / "X_train.parquet")
pd.DataFrame(
    X_val_p, columns=feature_names
).to_parquet(processed_dir / "X_validation.parquet")
pd.DataFrame(
    X_test_p, columns=feature_names
).to_parquet(processed_dir / "X_test.parquet")
y_train.to_frame("mean_power_w").to_parquet(
    processed_dir / "y_train.parquet"
)
y_val.to_frame("mean_power_w").to_parquet(
    processed_dir / "y_validation.parquet"
)
y_test.to_frame("mean_power_w").to_parquet(
    processed_dir / "y_test.parquet"
)
# Raw test features plus target, used for per workload evaluation in
03_Modeling.ipynb.
X_test.assign(
    mean_power_w=y_test
).to_parquet(
    processed_dir / "test_raw.parquet"
)
print(f"Saved to: {processed_dir}")
for file_path in sorted(processed_dir.iterdir()):
    print(f"{file_path.name}")
```

```
Saved to: /home/ec2-user/SageMaker/dataset-modeling
X_test.parquet
X_train.parquet
X_validation.parquet
preprocessor.joblib
test_raw.parquet
y_test.parquet
y_train.parquet
y_validation.parquet
```

```
from pathlib import Path
import pandas as pd
import numpy as np
# Load the processed datasets created by 02_Preprocessing.ipynb.
processed_dir = Path(
    "/home/ec2-user/SageMaker/dataset-modeling"
)
if not processed_dir.exists():
    raise FileNotFoundError(
```

```
        f"Processed modeling directory not found: {processed_dir}"
    )
X_train = pd.read_parquet(
    processed_dir / "X_train.parquet"
)
X_val = pd.read_parquet(
    processed_dir / "X_validation.parquet"
)
X_test = pd.read_parquet(
    processed_dir / "X_test.parquet"
)
y_train = pd.read_parquet(
    processed_dir / "y_train.parquet"
)["mean_power_w"]
y_val = pd.read_parquet(
    processed_dir / "y_validation.parquet"
)["mean_power_w"]
y_test = pd.read_parquet(
    processed_dir / "y_test.parquet"
)["mean_power_w"]
test_raw = pd.read_parquet(
    processed_dir / "test_raw.parquet"
)
print(
    f"Train:{X_train.shape}"
    f"\nValidation:{X_val.shape}"
    f"\nTest:{X_test.shape}"
)
print(f"\nProcessed feature count: {X_train.shape[1]}")
print(f"Target: mean_power_w")
```

```
Train:(1696, 14)
Validation:(364, 14)
Test:(364, 14)

Processed feature count: 14
Target: mean_power_w
```

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)
def evaluate(model, X, y, label):
    """Evaluate a regression model and return standard metrics."""
    predictions = model.predict(X)
    rmse = mean_squared_error(y, predictions) ** 0.5
    mae = mean_absolute_error(y, predictions)
    r2 = r2_score(y, predictions)
    mae_relative = mae / y.mean()
```

```
    print(
        f"{label:30s} "
        f"RMSE={rmse:8.2f} W | "
        f"MAE={mae:8.2f} W | "
        f"R²={r2:6.4f} | "
        f"MAE/mean={mae_relative:.2%}"
    )
    return {
        "model": label,
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2,
    }

from sklearn.dummy import DummyRegressor
results = []
# Baseline 1: predict the overall training set mean.
global_mean = DummyRegressor(
    strategy="mean"
).fit(
    X_train,
    y_train
)
results.append(
    evaluate(
        global_mean,
        X_val,
        y_val,
        "Baseline: global mean"
    )
)
# Baseline 2: predict the training set mean for each workload type.
workload_cols = [
    column
    for column in X_train.columns
    if column.startswith("categorical__workload_type_")
]
train_workload = (
    X_train[workload_cols]
    .idxmax(axis=1)
    .str.replace(
        "categorical__workload_type_",
        "",
        regex=False,
    )
)
workload_means = (
    y_train.groupby(train_workload.values)
    .mean()
)
```

```
class WorkloadMeanPredictor:
    """Predict the training set mean power for each workload type."""
    def predict(self, X):
        workload = (
            X[workload_cols]
            .idxmax(axis=1)
            .str.replace(
                "categorical__workload_type_",
                "",
                regex=False,
            )
        )
        return (
            workload
            .map(workload_means)
            .fillna(y_train.mean())
            .values
        )
workload_mean = WorkloadMeanPredictor()
results.append(
    evaluate(
        workload_mean,
        X_val,
        y_val,
        "Baseline: workload mean"
    )
)
```

```
Baseline: global mean          RMSE=  187.11 W | MAE=  150.29 W | R²=-
0.0002 | MAE/mean=5.79%
Baseline: workload mean        RMSE=  167.52 W | MAE=  115.37 W |
R²=0.1983 | MAE/mean=4.44%
```

```
from sklearn.linear_model import LinearRegression
linear_regression = LinearRegression().fit(
    X_train,
    y_train
)
results.append(
    evaluate(
        linear_regression,
        X_val,
        y_val,
        "Linear Regression"
    )
)
```

```
Linear Regression              RMSE=  146.84 W | MAE=  106.59 W |
R²=0.3840 | MAE/mean=4.11%
```

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV
random_forest = RandomForestRegressor(
    random_state=42,
    n_jobs=-1,
)
random_forest_params = {
    "n_estimators": [300, 500, 800],
    "max_depth": [None, 10, 20, 30],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", None],
}
random_forest_search = RandomizedSearchCV(
    estimator=random_forest,
    param_distributions=random_forest_params,
    n_iter=20,
    cv=5,
    scoring="neg_root_mean_squared_error",
    random_state=42,
    n_jobs=-1,
    verbose=1,
)
random_forest_search.fit(
    X_train,
    y_train,
)
print(
    f"Best parameters: "
    f"{random_forest_search.best_params_}"
)
print(
    f"Best CV RMSE: "
    f"{-random_forest_search.best_score_:.2f} W"
)
results.append(
    evaluate(
        random_forest_search.best_estimator_,
        X_val,
        y_val,
        "Random Forest (tuned)"
    )
)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best parameters: {'n_estimators': 300, 'min_samples_split': 2,
'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': 10}
Best CV RMSE: 39.72 W
Random Forest (tuned)          RMSE=   37.20 W | MAE=   24.74 W |
R²=0.9605 | MAE/mean=0.95%
```

```python
from xgboost import XGBRegressor
xgboost_model = XGBRegressor(
    random_state=42,
    n_jobs=-1,
)
xgboost_params = {
    "n_estimators": [500, 1000, 2000],
    "learning_rate": [0.01, 0.05, 0.1],
    "max_depth": [3, 4, 6, 8],
    "subsample": [0.7, 0.8, 1.0],
    "colsample_bytree": [0.7, 0.8, 1.0],
    "reg_lambda": [0.1, 1.0, 10.0],
}
xgboost_search = RandomizedSearchCV(
    estimator=xgboost_model,
    param_distributions=xgboost_params,
    n_iter=20,
    cv=5,
    scoring="neg_root_mean_squared_error",
    random_state=42,
    n_jobs=-1,
    verbose=1,
)
xgboost_search.fit(
    X_train,
    y_train,
)
print(
    f"Best parameters: "
    f"{xgboost_search.best_params_}"
)
print(
    f"Best CV RMSE: "
    f"{-xgboost_search.best_score_:.2f} W"
)
results.append(
    evaluate(
        xgboost_search.best_estimator_,
        X_val,
        y_val,
        "XGBoost (tuned)"
    )
)
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best parameters: {'subsample': 1.0, 'reg_lambda': 0.1, 'n_estimators':
1000, 'max_depth': 3, 'learning_rate': 0.05, 'colsample_bytree': 1.0}
Best CV RMSE: 39.27 W
XGBoost (tuned)                RMSE=   37.25 W | MAE=   25.27 W |
R²=0.9604 | MAE/mean=0.97%
```

```python
# Compare all candidate models using the validation set.
comparison = (
    pd.DataFrame(results)
    .sort_values("RMSE")
    .reset_index(drop=True)
)
display(comparison)
# Store every candidate model so the model selected from the
# validation results is always the model that is actually evaluated.
candidate_models = {
    "Baseline: global mean": global_mean,
    "Baseline: workload mean": workload_mean,
    "Linear Regression": linear_regression,
    "Random Forest (tuned)": random_forest_search.best_estimator_,
    "XGBoost (tuned)": xgboost_search.best_estimator_,
}
# Select the model with the lowest validation RMSE.
best_name = comparison.iloc[0]["model"]
best_model = candidate_models[best_name]
print(f"\nBest model on validation: {best_name}")
```

	model	RMSE	MAE	R2
0	Random Forest (tuned)	37.196192	24.740965	0.960473
1	XGBoost (tuned)	37.249581	25.273876	0.960360
2	Linear Regression	146.839116	106.585422	0.384005
3	Baseline: workload mean	167.521414	115.370299	0.198258
4	Baseline: global mean	187.113614	150.288525	-0.000241

```
Best model on validation: Random Forest (tuned)
```

```python
test_metrics = evaluate(
    best_model,
    X_test,
    y_test,
    f"TEST: {best_name}"
)
```

```
TEST: Random Forest (tuned)    RMSE=   37.61 W | MAE=   25.10 W |
R²=0.9599 | MAE/mean=0.97%
```

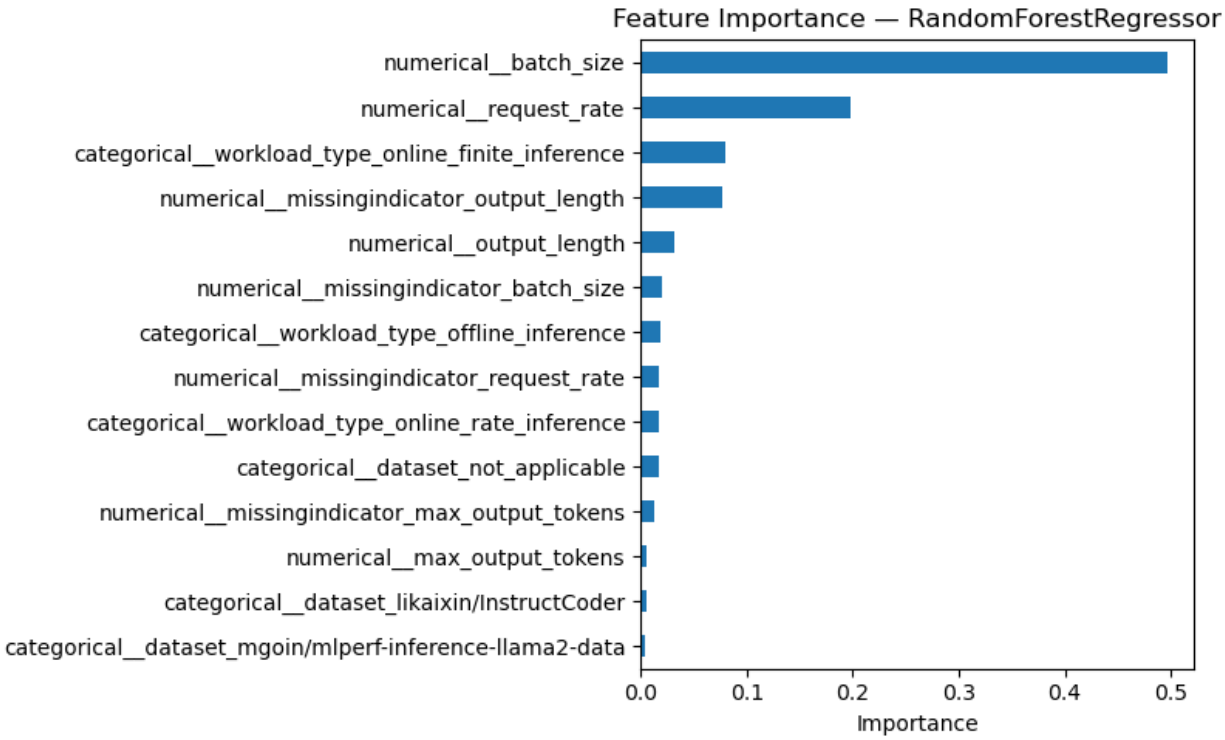
```python
predictions = best_model.predict(X_test)
per_workload = []
for workload_type in sorted(
    test_raw["workload_type"].unique()
):
    mask = (
        test_raw["workload_type"] == workload_type
    ).values
    per_workload.append({
```

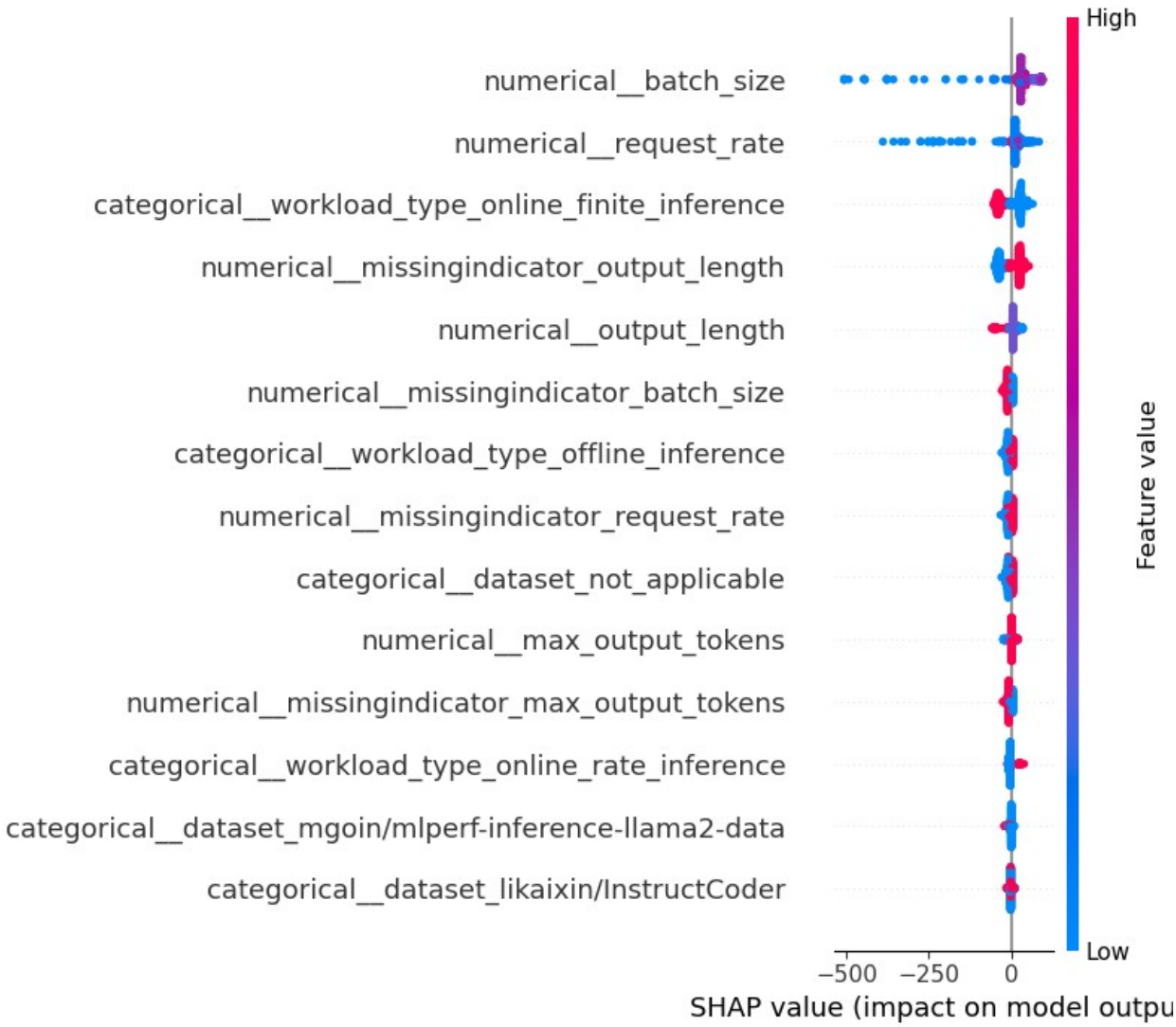
```
        "workload_type": workload_type,
        "n": int(mask.sum()),
        "RMSE": (
            mean_squared_error(
                y_test[mask],
                predictions[mask],
            ) ** 0.5
        ),
        "MAE": mean_absolute_error(
            y_test[mask],
            predictions[mask],
        ),
        "R2": r2_score(
            y_test[mask],
            predictions[mask],
        ),
    })
per_workload_results = pd.DataFrame(
    per_workload
)
display(per_workload_results)
```

	workload_type	n	RMSE	MAE	R2
0	offline_inference	180	40.789619	23.243548	0.953376
1	online_finite_inference	154	36.700295	29.856282	0.878908
2	online_rate_inference	30	16.279240	11.789266	0.994520

```
import matplotlib.pyplot as plt
# Feature importance is available for tree-based models.
if hasattr(best_model, "feature_importances_"):
    importance = (
        pd.Series(
            best_model.feature_importances_,
            index=X_train.columns,
        )
        .sort_values()
    )
    ax = importance.plot.barh(
        figsize=(8, 5),
        title=(
            f"Feature Importance — "
            f"{type(best_model).__name__}"
        ),
    )
    ax.set_xlabel("Importance")
    plt.tight_layout()
    plt.show()
else:
    print(
        "The selected model does not provide "
```

```
        "built in feature importances."
    )
# Attempt SHAP analysis for tree based models.
try:
    import shap
    if hasattr(best_model, "feature_importances_"):
        explainer = shap.TreeExplainer(best_model)
        shap_values = explainer.shap_values(
            X_test
        )
        shap.summary_plot(
            shap_values,
            X_test,
            feature_names=X_train.columns.tolist(),
            show=False,
        )
        plt.tight_layout()
        plt.show()
    else:
        print(
            "SHAP tree explanations are skipped "
            "because the selected model is not tree based."
        )
except ImportError:
    print(
        "SHAP is not installed. "
        "Install it with `pip install shap` if needed."
    )
```


Feature Importance — RandomForestRegressor
numerical__batch_size
numerical__request_rate
categorical__workload_type_online_finite_inference
numerical__missingindicator_output_length
numerical__output_length
numerical__missingindicator_batch_size
categorical__workload_type_offline_inference
numerical__missingindicator_request_rate
categorical__workload_type_online_rate_inference
categorical__dataset_not_applicable
numerical__missingindicator_max_output_tokens
numerical__max_output_tokens
categorical__dataset_likaixin/InstructCoder
categorical__dataset_mgoin/mlperf-inference-llama2-data
0.0
0.1
0.2
0.3
0.4
0.5
Importance




```
import joblib
model_path = processed_dir / "best_model.joblib"
model_artifact = {
    "model": best_model,
    "feature_names": X_train.columns.tolist(),
    "validation_metrics": comparison.iloc[0].to_dict(),
}
joblib.dump(
    model_artifact,
    model_path,
)
print(
    f"Selected model saved to:\n{model_path}"
)
print(
```

```
    f"\nSelected model: {best_name}"
)
```

```
Selected model saved to:
/home/ec2-user/SageMaker/dataset-modeling/best_model.joblib

Selected model: Random Forest (tuned)
```

```
# Deployment preparation: verify that the selected model artifact and
fitted preprocessor exist and can be loaded.
from pathlib import Path
import joblib

artifact_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")

model_path = artifact_dir / "best_model.joblib"
preprocessor_path = artifact_dir / "preprocessor.joblib"

print("Model exists:", model_path.exists())
print("Preprocessor exists:", preprocessor_path.exists())

best_model = joblib.load(model_path)
preprocessor = joblib.load(preprocessor_path)

print("\nModel type:", type(best_model))
print("Preprocessor type:", type(preprocessor))
```

```
Model exists: True
Preprocessor exists: True

Model type: <class 'dict'>
Preprocessor type: <class
'sklearn.compose._column_transformer.ColumnTransformer'>
```

```
# Inspect the selected model and confirm the preprocessor's
transformed feature names and count.
# The Random Forest is trained on 14 transformed features, not the six
raw request fields.
print("Model:", best_model)

if hasattr(preprocessor, "get_feature_names_out"):
    feature_names = preprocessor.get_feature_names_out()
    print("\nNumber of processed features:", len(feature_names))
    print("Feature names:")
    print(list(feature_names))

print("\nModel input features:",
      getattr(best_model, "n_features_in_", "Not exposed"))
```

```
Model: {'model': RandomForestRegressor(max_depth=10,
max_features='log2', n_estimators=300,
```

```
                      n_jobs=-1, random_state=42), 'feature_names':
['numerical__batch_size', 'numerical__max_output_tokens',
'numerical__request_rate', 'numerical__output_length',
'numerical__missingindicator_batch_size',
'numerical__missingindicator_max_output_tokens',
'numerical__missingindicator_request_rate',
'numerical__missingindicator_output_length',
'categorical__workload_type_offline_inference',
'categorical__workload_type_online_finite_inference',
'categorical__workload_type_online_rate_inference',
'categorical__dataset_likaixin/InstructCoder',
'categorical__dataset_mgoin/mlperf-inference-llama2-data',
'categorical__dataset_not_applicable'], 'validation_metrics':
{'model': 'Random Forest (tuned)', 'RMSE': 37.19619247477418, 'MAE':
24.740965028429606, 'R2': 0.9604732640635285}}

Number of processed features: 14
Feature names:
['numerical__batch_size', 'numerical__max_output_tokens',
'numerical__request_rate', 'numerical__output_length',
'numerical__missingindicator_batch_size',
'numerical__missingindicator_max_output_tokens',
'numerical__missingindicator_request_rate',
'numerical__missingindicator_output_length',
'categorical__workload_type_offline_inference',
'categorical__workload_type_online_finite_inference',
'categorical__workload_type_online_rate_inference',
'categorical__dataset_likaixin/InstructCoder',
'categorical__dataset_mgoin/mlperf-inference-llama2-data',
'categorical__dataset_not_applicable']

Model input features: Not exposed

# Artifact consistency check: extract the estimator from the saved
dictionary and confirm feature order.
# Recompute test metrics from the saved estimator using the already-
preprocessed held-out test matrix.

import joblib
import numpy as np
from pathlib import Path
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score

artifact_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")

saved_artifact = joblib.load(artifact_dir / "best_model.joblib")
preprocessor = joblib.load(artifact_dir / "preprocessor.joblib")

# Extract the actual Random Forest from the saved dictionary
```

```
rf_model = saved_artifact["model"]
saved_feature_names = saved_artifact["feature_names"]

# Confirm the preprocessor's output feature order
processed_feature_names = list(preprocessor.get_feature_names_out())

assert saved_feature_names == processed_feature_names, (
    "Feature names or order do not match!"
)

assert X_test.shape[1] == len(saved_feature_names), (
    "X_test has an unexpected number of features!"
)

# Predict directly from the already-preprocessed test matrix
y_test_pred = rf_model.predict(X_test)

print("Model type:", type(rf_model).__name__)
print("Number of features:", len(saved_feature_names))
print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test,
y_test_pred)):.2f} W")
print(f"Test MAE:  {mean_absolute_error(y_test, y_test_pred):.2f} W")
print(f"Test R²:   {r2_score(y_test, y_test_pred):.4f}")
```

```
Model type: RandomForestRegressor
Number of features: 14
Test RMSE: 37.61 W
Test MAE:  25.10 W
Test R²:   0.9599
```

```
# Record the software versions used in this notebook; version
alignment matters when loading joblib artifacts.
import sklearn
import joblib
import sagemaker

print("scikit-learn:", sklearn.__version__)
print("joblib:", joblib.__version__)
print("SageMaker SDK:", sagemaker.__version__)
```

```
sagemaker.config INFO - Not applying SDK defaults from location:
/etc/xdg/sagemaker/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location:
/home/ec2-user/.config/sagemaker/config.yaml

/home/ec2-user/anaconda3/envs/pytorch/lib/python3.10/site-packages/
sagemaker/__init__.py:86: SageMakerV2DeprecationWarning: You are using
the SageMaker Python SDK v2, which is on path of deprecation. v3 is
the actively developed major version.
See
https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md
```

```
for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to
silence this warning.
  warn_v2_deprecation()
You are using the SageMaker Python SDK v2, which is on path of
deprecation. v3 is the actively developed major version.
See
https://github.com/aws/sagemaker-python-sdk/blob/master/migration.md
for the migration guide. Set SAGEMAKER_SUPPRESS_V2_WARNING=1 to
silence this warning.

scikit-learn: 1.4.2
joblib: 1.5.3
SageMaker SDK: 2.257.6
```

```
# Resolve the SageMaker execution role, AWS region, and default S3
bucket for deployment resources.
import sagemaker
import boto3

session = sagemaker.Session()
role = sagemaker.get_execution_role()
region = boto3.Session().region_name

print("Region:", region)
print("SageMaker session bucket:", session.default_bucket())
print("Execution role:", role)
```

```
Region: us-east-1
SageMaker session bucket: sagemaker-us-east-1-451813790848
Execution role: arn:aws:iam::451813790848:role/LabRole
```

```
# Inspect the fitted preprocessor's raw input schema (six columns) and
transformed output width (14 features).
import joblib
from pathlib import Path

base_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")
preprocessor = joblib.load(base_dir / "preprocessor.joblib")

print("Expected raw input columns:")
print(list(preprocessor.feature_names_in_))

print("\nNumber of raw input columns:")
print(len(preprocessor.feature_names_in_))

print("\nOutput feature count:")
print(len(preprocessor.get_feature_names_out()))
```

```
Expected raw input columns:
['workload_type', 'batch_size', 'max_output_tokens', 'request_rate',
'output_length', 'dataset']
```

```
Number of raw input columns:
6

Output feature count:
14
```

```
# Discover which managed scikit-learn inference image versions are
recognized by this SageMaker SDK.
# The selected image below is the 1.4-2 container, matching scikit-
learn 1.4.2.
import sagemaker
from sagemaker import image_uris

print("SageMaker SDK:", sagemaker.__version__)

for version in ["1.7-1", "1.4-2"]:
    try:
        uri = image_uris.retrieve(
            framework="sklearn",
            region="us-east-1",
            version=version
        )
        print(f"scikit-learn container {version}: {uri}")
    except Exception as e:
        print(f"Container {version} unavailable:", str(e)[:250])
```

```
SageMaker SDK: 2.257.6
Container 1.7-1 unavailable: Unsupported sklearn version: 1.7-1. You
may need to upgrade your SDK version (pip install -U sagemaker) for
newer sklearn versions. Supported sklearn version(s): 0.20.0, 0.23-1,
1.0-1, 1.2-1, 1.4-2.
scikit-learn container 1.4-2: 683313688378.dkr.ecr.us-east-
1.amazonaws.com/sagemaker-scikit-learn:1.4-2-cpu-py3
```

```
# Confirm that the regenerated artifacts exist under the expected
SageMaker path and inspect saved validation metrics.
import sklearn
import joblib
from pathlib import Path

base_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")

print("scikit-learn:", sklearn.__version__)

for name in ["best_model.joblib", "preprocessor.joblib"]:
    path = base_dir / name
    print(f"{name}: {'FOUND' if path.exists() else 'MISSING'}")

saved = joblib.load(base_dir / "best_model.joblib")
```

```
print("Saved model:", type(saved["model"]).__name__)
print("Validation metrics:", saved.get("validation_metrics"))
```

```
scikit-learn: 1.4.2
best_model.joblib: FOUND
preprocessor.joblib: FOUND
Saved model: RandomForestRegressor
Validation metrics: {'model': 'Random Forest (tuned)', 'RMSE':
37.19619247477418, 'MAE': 24.740965028429606, 'R2':
0.9604732640635285}
```

```
# Confirm the saved artifact structure and the raw input/output
feature dimensions before packaging.
import joblib
from pathlib import Path

base_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")

model_obj = joblib.load(base_dir / "best_model.joblib")
preprocessor = joblib.load(base_dir / "preprocessor.joblib")

print("Model object type:", type(model_obj).__name__)
print("Preprocessor inputs:", list(preprocessor.feature_names_in_))
print("Processed feature count:",
len(preprocessor.get_feature_names_out()))
```

```
Model object type: dict
Preprocessor inputs: ['workload_type', 'batch_size',
'max_output_tokens', 'request_rate', 'output_length', 'dataset']
Processed feature count: 14
```

```
# Create the custom inference handler and deployment copies of the
estimator and preprocessor.
# The handler accepts raw JSON records, applies the fitted
preprocessor once, and returns predicted mean power in watts.
from pathlib import Path

base_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")
deploy_dir = base_dir / "rf-inference-code"
deploy_dir.mkdir(exist_ok=True)

inference_code = r'''
import os
import json
import joblib
import pandas as pd

def model_fn(model_dir):
    model = joblib.load(os.path.join(model_dir, "model.joblib"))
    preprocessor = joblib.load(os.path.join(model_dir,
"preprocessor.joblib"))
```

```
    return {"model": model, "preprocessor": preprocessor}

def input_fn(request_body, content_type):
    if content_type != "application/json":
        raise ValueError("Content-Type must be application/json")
    payload = json.loads(request_body)
    records = payload["instances"] if isinstance(payload, dict) else
payload
    return pd.DataFrame(records)

def predict_fn(data, artifacts):
    preprocessor = artifacts["preprocessor"]
    model = artifacts["model"]
    expected = list(preprocessor.feature_names_in_)
    missing = [c for c in expected if c not in data.columns]
    if missing:
        raise ValueError(f"Missing raw input columns: {missing}")
    transformed = preprocessor.transform(data[expected])
    return {
        "predicted_mean_power_w": [
            float(v) for v in model.predict(transformed)
        ]
    }

def output_fn(prediction, accept):
    return json.dumps(prediction), "application/json"
'''

(deploy_dir / "inference.py").write_text(inference_code)

import joblib
job = joblib.load(base_dir / "best_model.joblib")
rf_model = job["model"]
preprocessor = joblib.load(base_dir / "preprocessor.joblib")

joblib.dump(rf_model, deploy_dir / "model.joblib")
joblib.dump(preprocessor, deploy_dir / "preprocessor.joblib")

print("Prepared model, preprocessor and inference script.")
```

```
Prepared model, preprocessor and inference script.
```

```
# Bundle the model estimator, preprocessor, and inference handler into
model.tar.gz for SageMaker to download.
import tarfile

archive_path = base_dir / "model.tar.gz"

with tarfile.open(archive_path, "w:gz") as tar:
    for name in ["model.joblib", "preprocessor.joblib",
"inference.py"]:
```

```
        tar.add(deploy_dir / name, arcname=name)

print("Archive:", archive_path)
print("Size:", f"{archive_path.stat().st_size:,} bytes")
```

```
Archive: /home/ec2-user/SageMaker/dataset-modeling/model.tar.gz
Size: 2,414,797 bytes
```

```
# Upload the model archive to S3; SageMaker references this S3 URI
when creating the model resource.
import boto3
from pathlib import Path

region = "us-east-1"
bucket = "sagemaker-us-east-1-451813790848"
s3_key =
"prj-inference-power-feature-store/model/random-forest/model.tar.gz"

archive_path = Path(
    "/home/ec2-user/SageMaker/dataset-modeling/model.tar.gz"
)

s3 = boto3.client("s3", region_name=region)

s3.upload_file(
    str(archive_path),
    bucket,
    s3_key
)

model_data = f"s3://{bucket}/{s3_key}"
print("Model package uploaded:", model_data)
```

```
Model package uploaded: s3://sagemaker-us-east-1-451813790848/prj-
inference-power-feature-store/model/random-forest/model.tar.gz
```

```
# Upload inference.py separately as a source archive for the managed
scikit-learn serving container.
# The resulting source_uri is referenced in the model's environment
configuration in the next cell.
import tarfile
import boto3
from pathlib import Path

base_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")
deploy_dir = base_dir / "rf-inference-code"

source_archive = base_dir / "rf-inference-source.tar.gz"

# The source archive contains inference.py at its root.
with tarfile.open(source_archive, "w:gz") as tar:
```

```
    tar.add(deploy_dir / "inference.py", arcname="inference.py")

bucket = "sagemaker-us-east-1-451813790848"
source_key =
"prj-inference-power-feature-store/model/random-forest/rf-inference-
source.tar.gz"

s3 = boto3.client("s3", region_name="us-east-1")
s3.upload_file(str(source_archive), bucket, source_key)

source_uri = f"s3://{bucket}/{source_key}"
print("Inference source uploaded:", source_uri)
```

```
Inference source uploaded: s3://sagemaker-us-east-1-451813790848/prj-
inference-power-feature-store/model/random-forest/rf-inference-
source.tar.gz
```

```
# Create a SageMaker model resource using the managed scikit-learn
1.4-2 image, S3 model artifact, and inference source archive.
# This creates a model definition only; it does not start endpoint
hosting by itself.
import boto3
import sagemaker
from sagemaker import image_uris

region = "us-east-1"
role = sagemaker.get_execution_role()
sm = boto3.client("sagemaker", region_name=region)

image_uri = image_uris.retrieve(
    framework="sklearn",
    region=region,
    version="1.4-2",
    py_version="py3",
    instance_type="ml.m5.large",
)

bucket = "sagemaker-us-east-1-451813790848"
model_data = (
    f"s3://{bucket}/prj-inference-power-feature-store/"
    "model/random-forest/model.tar.gz"
)

model_name = "rf-power-prediction-142-v2-latest"

sm.create_model(
    ModelName=model_name,
    ExecutionRoleArn=role,
    PrimaryContainer={
        "Image": image_uri,
```

```
        "ModelDataUrl": model_data,
        "Environment": {
            "SAGEMAKER_PROGRAM": "inference.py",
            "SAGEMAKER_SUBMIT_DIRECTORY": source_uri,
        },
    },
)

print("Created model:", model_name)
print("Image:", image_uri)
```

```
Created model: rf-power-prediction-142-v2-latest
Image: 683313688378.dkr.ecr.us-east-1.amazonaws.com/sagemaker-scikit-learn:1.4-2-cpu-py3
```

```
# Diagnostic check for the created model
import boto3

sm = boto3.client("sagemaker", region_name="us-east-1")

response = sm.describe_model(
    ModelName="rf-power-prediction-142-v2-latest"
)

print("Model:", response["ModelName"])
print("Image:", response["PrimaryContainer"]["Image"])
print("Artifact:", response["PrimaryContainer"]["ModelDataUrl"])
print("Role:", response["ExecutionRoleArn"])
```

```
Model: rf-power-prediction-142-v2-latest
Image: 683313688378.dkr.ecr.us-east-1.amazonaws.com/sagemaker-scikit-learn:1.4-2-cpu-py3
Artifact: s3://sagemaker-us-east-1-451813790848/prj-inference-power-feature-store/model/random-forest/model.tar.gz
Role: arn:aws:iam::451813790848:role/LabRole
```

```
# Create an endpoint configuration and endpoint for the Random Forest model.
# This endpoint uses one ml.m5.large instance.
import boto3
import time

region = "us-east-1"
sm = boto3.client("sagemaker", region_name=region)

model_name = "rf-power-prediction-142-v2-latest"
endpoint_config_name = "rf-power-prediction-config-v2-latest"
endpoint_name = "rf-power-prediction-142-v2-latest"

# Create endpoint configuration
sm.create_endpoint_config(
```

```
    EndpointConfigName=endpoint_config_name,
    ProductionVariants=[
        {
            "VariantName": "AllTraffic",
            "ModelName": model_name,
            "InitialInstanceCount": 1,
            "InstanceType": "ml.m5.large",
            "InitialVariantWeight": 1.0,
        }
    ],
)

print("Endpoint configuration created:", endpoint_config_name)

# Create endpoint
sm.create_endpoint(
    EndpointName=endpoint_name,
    EndpointConfigName=endpoint_config_name,
)

print("Endpoint creation started:", endpoint_name)
```

```
Endpoint configuration created: rf-power-prediction-config-v2-latest
Endpoint creation started: rf-power-prediction-142-v2-latest
```

```
# Check if the Endpoint becomes Active
import boto3
import time

sm = boto3.client("sagemaker", region_name="us-east-1")
endpoint_name = "rf-power-prediction-142-v2-latest"

while True:
    response = sm.describe_endpoint(EndpointName=endpoint_name)
    status = response["EndpointStatus"]
    print("Endpoint status:", status)

    if status in ("InService", "Failed", "OutOfService"):
        if status == "Failed":
            print("Failure reason:", response.get("FailureReason"))
        break

    time.sleep(30)
```

```
Endpoint status: Creating
Endpoint status: Creating
Endpoint status: Creating
Endpoint status: Creating
Endpoint status: Creating
Endpoint status: Creating
Endpoint status: InService
```

```
# Smoke test: invoke the live endpoint with one example raw workload
record to verify request/response handling.
# This is a functional check, not an accuracy evaluation.
import boto3
import json

runtime = boto3.client("sagemaker-runtime", region_name="us-east-1")

endpoint_name = "rf-power-prediction-142-v2-latest"

sample = {
    "instances": [
        {
            "workload_type": "offline_inference",
            "batch_size": 1,
            "max_output_tokens": 128,
            "request_rate": None,
            "output_length": 100,
            "dataset": "not_applicable"
        }
    ]
}

response = runtime.invoke_endpoint(
    EndpointName=endpoint_name,
    ContentType="application/json",
    Body=json.dumps(sample)
)

result = json.loads(response["Body"].read().decode("utf-8"))
print("Live prediction:", result)
```

```
Live prediction: {'predicted_mean_power_w': [2142.5102202131675]}
```

```
# End-to-end consistency check: send a held-out raw test row to the
notebook model and live endpoint.
# Matching predictions verify that preprocessing and model inference
are aligned for this example.

import boto3
import json
import joblib
import pandas as pd
import numpy as np
from pathlib import Path

base_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")
endpoint_name = "rf-power-prediction-142-v2"

# Load regenerated artifacts
```

```
saved = joblib.load(base_dir / "best_model.joblib")
rf_model = saved["model"]
preprocessor = joblib.load(base_dir / "preprocessor.joblib")

# Read one held-out raw test example
test_raw = pd.read_parquet(base_dir / "test_raw.parquet")
raw_columns = list(preprocessor.feature_names_in_)
sample = test_raw[raw_columns].iloc[[0]]

# Notebook prediction: preprocess raw inputs, then predict
X_sample = preprocessor.transform(sample)
notebook_prediction = float(rf_model.predict(X_sample)[0])

# Endpoint prediction: send the exact same raw inputs
runtime = boto3.client("sagemaker-runtime", region_name="us-east-1")
payload = {"instances": sample.astype(object).where(
    pd.notna(sample), None
).to_dict(orient="records")}

response = runtime.invoke_endpoint(
    EndpointName=endpoint_name,
    ContentType="application/json",
    Body=json.dumps(payload)
)

endpoint_result = json.loads(response["Body"].read().decode("utf-8"))
endpoint_prediction = float(
    endpoint_result["predicted_mean_power_w"][0]
)

print("Notebook prediction:", round(notebook_prediction, 4), "W")
print("Endpoint prediction:", round(endpoint_prediction, 4), "W")
print("Absolute difference:", round(abs(
    notebook_prediction - endpoint_prediction
), 6), "W")
```

```
/home/ec2-user/anaconda3/envs/pytorch/lib/python3.10/site-packages/
sklearn/base.py:493: UserWarning: X does not have valid feature names,
but RandomForestRegressor was fitted with feature names
  warnings.warn(

Notebook prediction: 2258.6723 W
Endpoint prediction: 2258.6723 W
Absolute difference: 0.0 W
```

Infrastructure Optimization and Power Aware Scheduling

This notebook extends the GenAI workload power prediction model into a simulated infrastructure decision layer.

The goal is to evaluate whether predicted workload power can support power aware scheduling and capacity planning decisions under hypothetical infrastructure constraints.

The scheduling results in this notebook are simulations and do not represent actual NLR production scheduling decisions or measured infrastructure savings.

Load the Trained Model and Generate Power Predictions

We load the saved Random Forest model that was deployed earlier and the held out test data from the local path.

The model predicts execution level mean power for each workload configuration. These predictions will be used as the input to the decision layer, where we will evaluate how power aware scheduling could support infrastructure capacity planning.

The actual power values are retained only for evaluating the quality of the simulated scheduling decisions. The scheduler itself will make decisions using predicted power rather than observed power.

```
from pathlib import Path
import joblib
import pandas as pd
# Paths to the persisted modeling artifacts.
processed_dir = Path("/home/ec2-user/SageMaker/dataset-modeling")
model_path = processed_dir / "best_model.joblib"
if not model_path.exists():
    raise FileNotFoundError(f"Trained model not found: {model_path}")
# Load the selected model artifact created in 03_Modeling.ipynb.
model_artifact = joblib.load(model_path)
best_model = model_artifact["model"]
feature_names = model_artifact["feature_names"]
# Load the held out test data used for model evaluation.
X_test = pd.read_parquet(processed_dir / "X_test.parquet")
y_test = pd.read_parquet(processed_dir / "y_test.parquet")
["mean_power_w"]
test_raw = pd.read_parquet(processed_dir / "test_raw.parquet")
print(f"Loaded model: {type(best_model).__name__}")
print(f"Test records: {len(X_test):,}")
print(f"Features: {len(feature_names)}")
```

```
Loaded model: RandomForestRegressor
Test records: 364
Features: 14
```

```python
# Check that the test data matches the feature schema used by the
trained model.
if list(X_test.columns) != feature_names:
    raise ValueError("Test feature columns do not match the saved
model feature names.")
print("Model and test features are aligned.")
print(f"Model: {type(best_model).__name__}")
print(f"Test records: {len(X_test):,}")
```

```
Model and test features are aligned.
Model: RandomForestRegressor
Test records: 364
```

```python
# Prediction Layer
# Generate predicted execution level mean power.
predicted_power_w = best_model.predict(X_test)
prediction_results = test_raw.copy()
prediction_results["actual_power_w"] = y_test.values
prediction_results["predicted_power_w"] = predicted_power_w
prediction_results[
    ["workload_type", "actual_power_w", "predicted_power_w"]
].head(10)
```

	workload_type	actual_power_w	predicted_power_w
2186	online_finite_inference	2256.377925	2258.672307
1793	online_finite_inference	2625.840706	2603.236250
1272	online_finite_inference	2541.145137	2553.955001
1775	online_finite_inference	2552.795470	2585.225585
1822	online_finite_inference	2551.016595	2536.985344
579	offline_inference	2129.736377	2116.597530
1684	online_finite_inference	2622.444863	2560.323443
1757	online_finite_inference	2476.994837	2470.451139
112	offline_inference	2690.920602	2673.913892
2231	online_rate_inference	2215.768588	2194.798167

```python
print("Prediction summary:")
print(f"Mean predicted power: {predicted_power_w.mean():.2f} W")
print(f"Minimum predicted power: {predicted_power_w.min():.2f} W")
print(f"Maximum predicted power: {predicted_power_w.max():.2f} W")
```

```
Prediction summary:
Mean predicted power: 2594.00 W
Minimum predicted power: 2017.56 W
Maximum predicted power: 2874.70 W
```

Define Infrastructure Constraints

To evaluate the model as a decision support tool, we simulate a data center environment with a fixed power budget and a limited number of available execution slots.

These constraints are hypothetical and are used to test whether predicted workload power can be incorporated into scheduling decisions. The values are not intended to represent the actual power capacity or scheduling policy of the NLR infrastructure.

```
# Define hypothetical infrastructure constraints for the scheduling simulation.
POWER_BUDGET_W = 10_000
AVAILABLE_SLOTS = 4
print(f"Power budget: {POWER_BUDGET_W:,} W")
print(f"Available execution slots: {AVAILABLE_SLOTS}")
```

```
Power budget: 10,000 W
Available execution slots: 4
```

Prepare Candidate Workloads

The scheduler needs a set of candidate workload executions to evaluate. We use the test set predictions generated above so that each candidate has an estimated power requirement.

The scheduler will use these predicted values to determine which workloads can be placed within the available power budget and execution capacity.

```
#Prepare candidate workloads for the scheduling simulation.
candidate_workloads = prediction_results[
    [
        "workload_type",
        "predicted_power_w",
        "actual_power_w",
    ]
].copy()
candidate_workloads["predicted_power_w"] = (
    candidate_workloads["predicted_power_w"].round(2)
)
candidate_workloads.head()
```

	workload_type	predicted_power_w	actual_power_w
2186	online_finite_inference	2258.67	2256.377925
1793	online_finite_inference	2603.24	2625.840706
1272	online_finite_inference	2553.96	2541.145137
1775	online_finite_inference	2585.23	2552.795470
1822	online_finite_inference	2536.99	2551.016595

Implement the Power Aware Scheduling Decision

The scheduler evaluates candidate workloads using their predicted power requirements, the available execution slots, and the infrastructure power budget.

For this initial simulation, a workload is scheduled only when adding its predicted power would keep the total scheduled power within the defined budget and an execution slot is available.

This is a deterministic scheduling rule used to demonstrate how the ML predictions can support an infrastructure decision. It is not intended to represent a production scheduling algorithm.

Establish a Baseline Scheduler

To measure the value of power aware scheduling, we need a simple baseline for comparison.

The baseline represents a scheduler that fills available execution slots without considering predicted power consumption. Its scheduled power is then evaluated against the same infrastructure power budget.

Comparing the two approaches allows us to quantify whether incorporating ML based power predictions improves power budget compliance and infrastructure utilization.

```
# Baseline: schedule the first available workloads without considering
power.
baseline_workloads = candidate_workloads.head(AVAILABLE_SLOTS).copy()
baseline_predicted_power_w =
baseline_workloads["predicted_power_w"].sum()
print(f"Baseline workloads scheduled: {len(baseline_workloads)}")
print(f"Baseline predicted power: {baseline_predicted_power_w:,.2f}
W")
print(
    f"Power budget exceeded: "
    f"{baseline_predicted_power_w > POWER_BUDGET_W}"
)
```

```
Baseline workloads scheduled: 4
Baseline predicted power: 10,001.10 W
Power budget exceeded: True
```

Create a Scheduling Batch

A meaningful scheduling comparison requires more candidate workloads than available execution slots.

We use a batch of candidate executions from the test set and allow the scheduler to select up to four workloads. This creates a decision scenario where multiple workload combinations are possible under the same power constraint.

The baseline will follow a first come, first served strategy, while the power aware scheduler will use predicted power to make its selection.

```
# Use a larger candidate batch so the scheduler has multiple choices.
SCHEDULING_BATCH_SIZE = len(candidate_workloads)   # use full test set
scheduling_batch =
candidate_workloads.head(SCHEDULING_BATCH_SIZE).copy()
print(f"Candidate workloads: {len(scheduling_batch)}")
print(f"Available execution slots: {AVAILABLE_SLOTS}")
print(f"Power budget: {POWER_BUDGET_W:,} W")
```

```
Candidate workloads: 364
Available execution slots: 4
Power budget: 10,000 W
```

Run the Power Aware Scheduling Strategy

The scheduler now evaluates multiple candidate workloads rather than processing only the first available executions.

At each scheduling step, it selects the candidate with the lowest predicted power that can fit within the remaining power budget. This provides a simple and transparent power aware strategy for the initial simulation.

The objective is to demonstrate how ML based power predictions can influence workload placement while respecting both infrastructure capacity and power constraints.

```
def power_aware_schedule(
    workloads,
    power_budget_w,
    available_slots,
    safety_margin_w=0,
):
    """
    Greedy, lowest predicted power first scheduler.
    NOTE: This is one simple heuristic, not a proven optimal
scheduling
    policy,it prioritizes minimizing scheduled power over maximizing
    throughput/utilization. safety_margin_w reserves headroom against
    prediction error (e.g. set it to the model's test MAE from
    03_Modeling.ipynb) so the *actual* power is less likely to exceed
    the budget even though the decision is made on *predicted* power.
    """
    remaining = workloads.copy()
    scheduled_indices = []
    total_predicted_power_w = 0.0
    effective_budget_w = power_budget_w - safety_margin_w
    while (
        len(scheduled_indices) < available_slots
        and not remaining.empty
    ):
        # Keep only workloads that fit within the remaining (margin
adjusted) budget.
        feasible = remaining[
            total_predicted_power_w + remaining["predicted_power_w"]
            <= effective_budget_w
        ]
        if feasible.empty:
            break
        # Select the lowest power feasible workload.
```

```
        selected_index = feasible["predicted_power_w"].idxmin()
        selected_power = remaining.loc[
            selected_index, "predicted_power_w"
        ]
        scheduled_indices.append(selected_index)
        total_predicted_power_w += selected_power
        remaining = remaining.drop(index=selected_index)
    return scheduled_indices, total_predicted_power_w
# Reserve a safety margin roughly equal to the model's test MAE (~25W,
from 03_Modeling.ipynb)
SAFETY_MARGIN_W = 25
power_aware_indices, power_aware_power_w = power_aware_schedule(
    scheduling_batch,
    POWER_BUDGET_W,
    AVAILABLE_SLOTS,
    safety_margin_w=SAFETY_MARGIN_W,
)
print(f"Power aware workloads scheduled: {len(power_aware_indices)}")
print(f"Predicted scheduled power: {power_aware_power_w:,.2f} W")
print(f"Safety margin reserved: {SAFETY_MARGIN_W:,.2f} W")
print(
    f"Remaining power capacity: "
    f"{POWER_BUDGET_W - power_aware_power_w:,.2f} W"
)
```

```
Power aware workloads scheduled: 4
Predicted scheduled power: 8,070.24 W
Safety margin reserved: 25.00 W
Remaining power capacity: 1,929.76 W
```

Compare Scheduling Strategies

The power aware scheduler selects workloads using predicted power, while the baseline follows a first come, first served strategy.

We compare both approaches using the same candidate batch and infrastructure constraints. After scheduling, the actual measured power is used only to evaluate how well each strategy performed against the power budget.

This allows us to quantify the practical value of incorporating ML based power predictions into infrastructure scheduling decisions.

```
# First come, first served baseline.
baseline_indices = scheduling_batch.head(AVAILABLE_SLOTS).index
baseline_selected = scheduling_batch.loc[baseline_indices]
baseline_predicted_power_w =
baseline_selected["predicted_power_w"].sum()
baseline_actual_power_w = baseline_selected["actual_power_w"].sum()
print(f"Baseline workloads scheduled: {len(baseline_selected)}")
```

```
print(f"Baseline predicted power: {baseline_predicted_power_w:,.2f}
W")
print(f"Baseline actual power: {baseline_actual_power_w:,.2f} W")
```

```
Baseline workloads scheduled: 4
Baseline predicted power: 10,001.10 W
Baseline actual power: 9,976.16 W
```

Evaluate the Power Aware Schedule

The scheduler made its decision using predicted power only. We now evaluate the selected workloads using their actual measured power from the test dataset.

This separation is important: actual power is not used to make the scheduling decision. It is used only after scheduling to determine whether the prediction driven decision stayed within the simulated infrastructure constraints.

```
power_aware_selected = scheduling_batch.loc[power_aware_indices]
power_aware_actual_power_w = (
    power_aware_selected["actual_power_w"].sum()
)
print(f"Power aware workloads scheduled: {len(power_aware_selected)}")
print(f"Predicted power: {power_aware_power_w:,.2f} W")
print(f"Actual power: {power_aware_actual_power_w:,.2f} W")
print(
    f"Actual remaining capacity: "
    f"{POWER_BUDGET_W - power_aware_actual_power_w:,.2f} W"
)
```

```
Power aware workloads scheduled: 4
Predicted power: 8,070.24 W
Actual power: 8,089.16 W
Actual remaining capacity: 1,910.84 W
```

Quantify Scheduling Performance

We compare the baseline and power aware strategies using the same infrastructure constraints.

The key measures are scheduled workload count, actual power consumption, power budget utilization, remaining capacity, and whether the schedule exceeded the available power budget.

These metrics provide a direct measure of how the prediction model can support a measurable infrastructure decision.

```
baseline_utilization = (
    baseline_actual_power_w / POWER_BUDGET_W
)
power_aware_utilization = (
    power_aware_actual_power_w / POWER_BUDGET_W
```

```
)
scheduling_comparison = pd.DataFrame([
    {
        "strategy": "FCFS baseline",
        "workloads_scheduled": len(baseline_selected),
        "actual_power_w": baseline_actual_power_w,
        "power_utilization": baseline_utilization,
        "remaining_capacity_w": POWER_BUDGET_W -
baseline_actual_power_w,
        "budget_violation": baseline_actual_power_w > POWER_BUDGET_W,
    },
    {
        "strategy": "Power-aware",
        "workloads_scheduled": len(power_aware_selected),
        "actual_power_w": power_aware_actual_power_w,
        "power_utilization": power_aware_utilization,
        "remaining_capacity_w": POWER_BUDGET_W -
power_aware_actual_power_w,
        "budget_violation": power_aware_actual_power_w >
POWER_BUDGET_W,
    },
])
display(scheduling_comparison)
```

```
        strategy  workloads_scheduled  actual_power_w
power_utilization  \
0  FCFS baseline                    4     9976.159238
0.997616
1    Power-aware                    4     8089.163473
0.808916

   remaining_capacity_w  budget_violation
0             23.840762             False
1           1910.836527             False
```

Measure the Infrastructure Improvement

The power aware scheduler achieved the same number of scheduled workloads while using less power than the FCFS baseline.

We calculate the relative reduction in actual power consumption and the additional power capacity made available by the power aware strategy.

```
#Calculate improvement
power_reduction_w = (
    baseline_actual_power_w - power_aware_actual_power_w
)
power_reduction_pct = (
    power_reduction_w / baseline_actual_power_w * 100
```

```
)
additional_headroom_w = (
    (POWER_BUDGET_W - power_aware_actual_power_w)
    - (POWER_BUDGET_W - baseline_actual_power_w)
)
print(f"Power reduction: {power_reduction_w:,.2f} W")
print(f"Relative power reduction: {power_reduction_pct:.2f}%")
print(f"Additional power headroom: {additional_headroom_w:,.2f} W")
```

```
Power reduction: 1,887.00 W
Relative power reduction: 18.92%
Additional power headroom: 1,887.00 W
```

Visualize the Scheduling Comparison

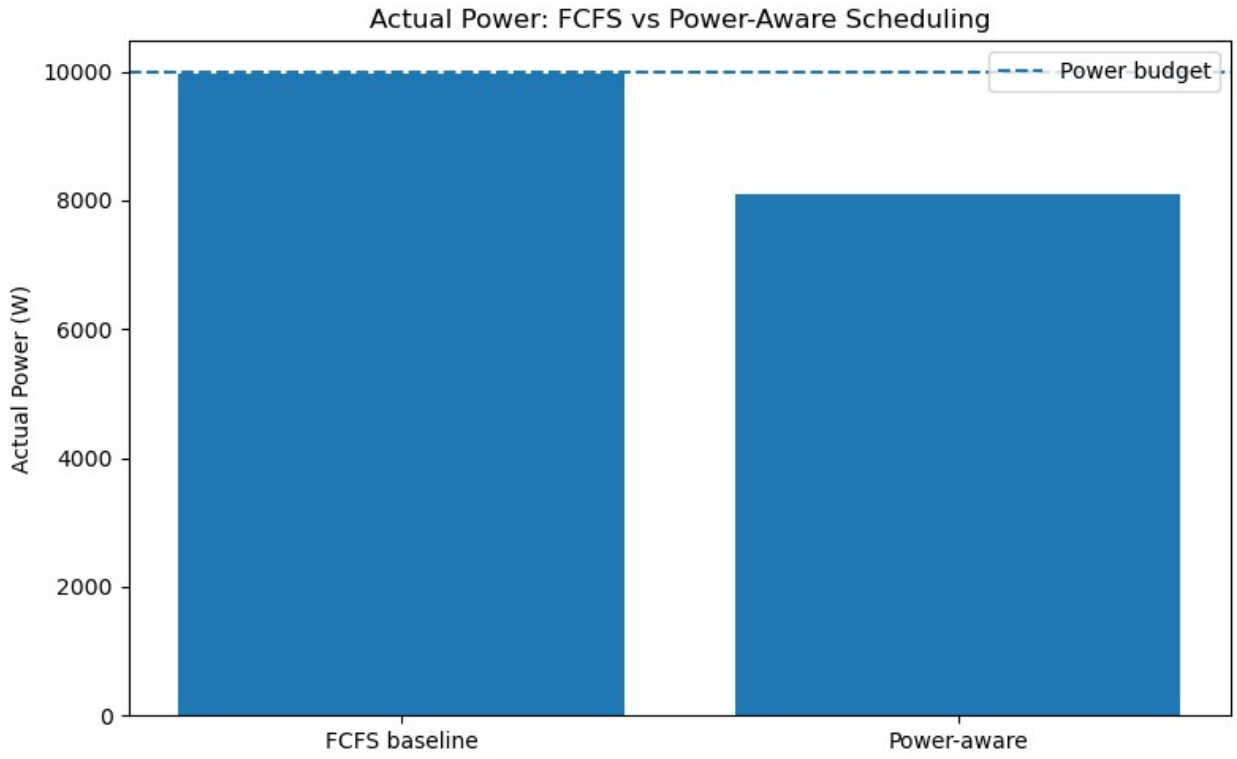
The following comparison shows how the two scheduling strategies use the same 10 kW simulated power budget.

The visualization highlights the reduction in actual power consumption achieved by the power aware strategy while maintaining the same number of scheduled workloads.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
plt.bar(
    scheduling_comparison["strategy"],
    scheduling_comparison["actual_power_w"],
)

plt.axhline(
    POWER_BUDGET_W,
    linestyle="--",
    label="Power budget",
)

plt.ylabel("Actual Power (W)")
plt.title("Actual Power: FCFS vs Power-Aware Scheduling")
plt.legend()
plt.tight_layout()
plt.show()
```



Evaluate Scheduling Across Multiple Power Budgets

A single power budget provides only one scheduling scenario. To make the comparison more robust, we evaluate both scheduling strategies across multiple hypothetical infrastructure power budgets.

The candidate workloads, execution slot limit, and scheduling rules remain unchanged. Only the available power budget varies.

For each budget, the baseline follows the first come, first served strategy, while the power aware scheduler selects workloads using predicted power. Actual measured power is used only after scheduling to evaluate the resulting decisions.

This experiment shows whether the observed scheduling improvement remains consistent across different infrastructure capacity constraints.

```
POWER_BUDGETS_W = [
    8_000,
    9_000,
    10_000,
    11_000,
    12_000,
]
multi_budget_results = []
for power_budget_w in POWER_BUDGETS_W:
    # FCFS baseline
```

```
    baseline_indices = scheduling_batch.head(AVAILABLE_SLOTS).index
    baseline_selected = scheduling_batch.loc[baseline_indices]
    baseline_predicted_power_w = (
        baseline_selected["predicted_power_w"].sum()
    )
    baseline_actual_power_w = (
        baseline_selected["actual_power_w"].sum()
    )
    # Power aware strategy
    power_aware_indices, power_aware_predicted_power_w = (
        power_aware_schedule(
            scheduling_batch,
            power_budget_w,
            AVAILABLE_SLOTS,
            safety_margin_w=SAFETY_MARGIN_W,
        )
    )
    power_aware_selected = scheduling_batch.loc[power_aware_indices]
    power_aware_actual_power_w = (
        power_aware_selected["actual_power_w"].sum()
    )
    # Store results for both strategies
    multi_budget_results.extend([
        {
            "power_budget_w": power_budget_w,
            "strategy": "FCFS baseline",
            "workloads_scheduled": len(baseline_selected),
            "predicted_power_w": baseline_predicted_power_w,
            "actual_power_w": baseline_actual_power_w,
            "power_utilization": (
                baseline_actual_power_w / power_budget_w
            ),
            "budget_violation": (
                baseline_actual_power_w > power_budget_w
            ),
        },
        {
            "power_budget_w": power_budget_w,
            "strategy": "Power-aware",
            "workloads_scheduled": len(power_aware_selected),
            "predicted_power_w": power_aware_predicted_power_w,
            "actual_power_w": power_aware_actual_power_w,
            "power_utilization": (
                power_aware_actual_power_w / power_budget_w
            ),
            "budget_violation": (
                power_aware_actual_power_w > power_budget_w
            ),
        },
```

```
    ])
multi_budget_comparison = pd.DataFrame(multi_budget_results)
display(multi_budget_comparison)
```

```
   power_budget_w       strategy  workloads_scheduled
predicted_power_w  \
0            8000  FCFS baseline                    4
10001.10
1            8000    Power-aware                    3
6052.68
2            9000  FCFS baseline                    4
10001.10
3            9000    Power-aware                    4
8070.24
4           10000  FCFS baseline                    4
10001.10
5           10000    Power-aware                    4
8070.24
6           11000  FCFS baseline                    4
10001.10
7           11000    Power-aware                    4
8070.24
8           12000  FCFS baseline                    4
10001.10
9           12000    Power-aware                    4
8070.24

   actual_power_w  power_utilization  budget_violation
0     9976.159238           1.247020              True
1     6077.111857           0.759639             False
2     9976.159238           1.108462              True
3     8089.163473           0.898796             False
4     9976.159238           0.997616             False
5     8089.163473           0.808916             False
6     9976.159238           0.906924             False
7     8089.163473           0.735378             False
8     9976.159238           0.831347             False
9     8089.163473           0.674097             False
```

Quantify Improvement Across Power Budgets

The multi budget experiment allows us to evaluate whether the scheduling benefit is consistent across different infrastructure constraints.

For each power budget, we calculate the actual remaining power capacity and the difference in actual power consumption between the FCFS baseline and the power aware strategy.

These results are used to determine whether power aware scheduling can reduce power consumption or prevent budget violations while maintaining as many scheduled workloads as possible.

```
budget_summary = (
    multi_budget_comparison
    .pivot(
        index="power_budget_w",
        columns="strategy",
        values=[
            "workloads_scheduled",
            "actual_power_w",
            "power_utilization",
            "budget_violation",
        ],
    )
)

budget_summary.columns = [
    f"{metric}_{strategy.replace(' ', '_').replace('-', '_').lower()}"
    for metric, strategy in budget_summary.columns
]

budget_summary = budget_summary.reset_index()

budget_summary["fcfs_headroom_w"] = (
    budget_summary["power_budget_w"]
    - budget_summary["actual_power_w_fcfs_baseline"]
)

budget_summary["power_aware_headroom_w"] = (
    budget_summary["power_budget_w"]
    - budget_summary["actual_power_w_power_aware"]
)

budget_summary["power_reduction_w"] = (
    budget_summary["actual_power_w_fcfs_baseline"]
    - budget_summary["actual_power_w_power_aware"]
)

budget_summary["power_reduction_pct"] = (
    budget_summary["power_reduction_w"]
    / budget_summary["actual_power_w_fcfs_baseline"]
    * 100
)

display(
    budget_summary[
        [
            "power_budget_w",
            "workloads_scheduled_fcfs_baseline",
            "workloads_scheduled_power_aware",
            "actual_power_w_fcfs_baseline",
            "actual_power_w_power_aware",
```

```
            "fcfs_headroom_w",
            "power_aware_headroom_w",
            "power_reduction_w",
            "power_reduction_pct",
            "budget_violation_fcfs_baseline",
            "budget_violation_power_aware",
        ]
    ]
)
```

```
   power_budget_w workloads_scheduled_fcfs_baseline  \
0            8000                                 4
1            9000                                 4
2           10000                                 4
3           11000                                 4
4           12000                                 4

  workloads_scheduled_power_aware actual_power_w_fcfs_baseline  \
0                               3                  9976.159238
1                               4                  9976.159238
2                               4                  9976.159238
3                               4                  9976.159238
4                               4                  9976.159238

  actual_power_w_power_aware fcfs_headroom_w power_aware_headroom_w  \
0                6077.111857    -1976.159238            1922.888143
1                8089.163473     -976.159238             910.836527
2                8089.163473       23.840762            1910.836527
3                8089.163473     1023.840762            2910.836527
4                8089.163473     2023.840762            3910.836527

  power_reduction_w power_reduction_pct budget_violation_fcfs_baseline
\
0       3899.047381           39.083652                           True

1       1886.995765           18.915053                           True

2       1886.995765           18.915053                          False

3       1886.995765           18.915053                          False

4       1886.995765           18.915053                          False

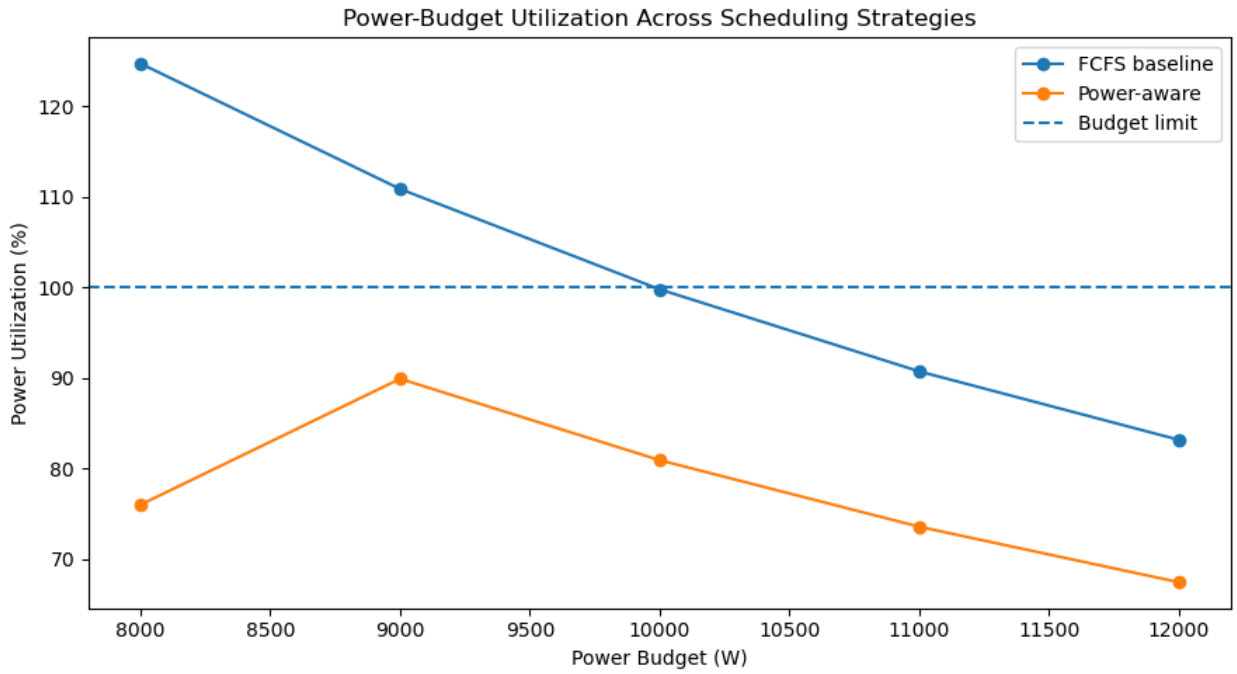

  budget_violation_power_aware
0                        False
1                        False
2                        False
3                        False
4                        False
```

Visualize Power Budget Utilization

The following visualization compares power budget utilization for the FCFS baseline and power aware scheduling strategies across multiple hypothetical infrastructure power budgets.

A utilization above 100% indicates that the schedule exceeded the available power budget. The comparison shows how predicted power can help the scheduler remain within tighter infrastructure constraints while maintaining workload capacity when feasible.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(9, 5))
for strategy in ["FCFS baseline", "Power-aware"]:
    strategy_data = multi_budget_comparison[
        multi_budget_comparison["strategy"] == strategy
    ]
    plt.plot(
        strategy_data["power_budget_w"],
        strategy_data["power_utilization"] * 100,
        marker="o",
        label=strategy,
    )
plt.axhline(
    100,
    linestyle="--",
    label="Budget limit",
)
plt.xlabel("Power Budget (W)")
plt.ylabel("Power Utilization (%)")
plt.title("Power-Budget Utilization Across Scheduling Strategies")
plt.legend()
plt.tight_layout()
plt.show()
```




```
# Create clean column names for easier downstream analysis.
budget_summary = budget_summary.rename(
    columns={
        "workloads_scheduled_fcfs_baseline": "fcfs_workloads",
        "workloads_scheduled_power_aware": "power_aware_workloads",
        "actual_power_w_fcfs_baseline": "fcfs_actual_power_w",
        "actual_power_w_power_aware": "power_aware_actual_power_w",
        "budget_violation_fcfs_baseline": "fcfs_budget_violation",
        "budget_violation_power_aware":
"power_aware_budget_violation",
    }
)

# Find the maximum power reduction when both strategies
# schedule the same number of workloads.
equal_workload_results = budget_summary[
    budget_summary["fcfs_workloads"]
    == budget_summary["power_aware_workloads"]
]

max_equal_workload_reduction = (
    equal_workload_results["power_reduction_pct"].max()
)

# Retrieve the 8 kW, 9 kW, and 10 kW scenarios.
eight_kw_result = budget_summary[
    budget_summary["power_budget_w"] == 8_000
].iloc[0]
```

```
nine_kw_result = budget_summary[
    budget_summary["power_budget_w"] == 9_000
].iloc[0]

ten_kw_result = budget_summary[
    budget_summary["power_budget_w"] == 10_000
].iloc[0]

print("Infrastructure Optimization Summary")
print("-" * 40)

print(
    f"Across the full 364 execution test set, at 9 kW, "
    f"power aware scheduling selected "
    f"{int(nine_kw_result['power_aware_workloads'])} workloads "
    f"without exceeding the power budget, while FCFS exceeded the
budget."
)

print(
    f"At 8 kW, power aware scheduling selected "
    f"{int(eight_kw_result['power_aware_workloads'])} workloads "
    f"compared with {int(eight_kw_result['fcfs_workloads'])} for FCFS
"
    f"while remaining within the power constraint."
)

print(
    f"When both strategies selected the same number of workloads, "
    f"the maximum observed reduction in actual scheduled power was "
    f"{max_equal_workload_reduction:.2f}%."
)

print(
    f"At the 10 kW budget, power-aware scheduling provided "
    f"{ten_kw_result['power_aware_headroom_w']:,.0f} W "
    f"of remaining capacity compared with "
    f"{ten_kw_result['fcfs_headroom_w']:,.0f} W for FCFS."
)
```

```
Infrastructure Optimization Summary
----------------------------------------
Across the full 364 execution test set, at 9 kW, power aware
scheduling selected 4 workloads without exceeding the power budget,
while FCFS exceeded the budget.
At 8 kW, power aware scheduling selected 3 workloads compared with 4
for FCFS while remaining within the power constraint.
When both strategies selected the same number of workloads, the
maximum observed reduction in actual scheduled power was 18.92%.
```

```
At the 10 kW budget, power-aware scheduling provided 1,911 W of
remaining capacity compared with 24 W for FCFS.
```

Infrastructure Optimization Summary

Across the full 364 execution test set, at 9 kW, power aware scheduling selected 4 workloads without exceeding the power budget, while FCFS exceeded the budget. At 8 kW, power aware scheduling selected 3 workloads compared with 4 for FCFS while remaining within the power constraint. When both strategies selected the same number of workloads, the maximum observed reduction in actual scheduled power was 18.92%. At the 10 kW budget, power-aware scheduling provided 1,911 W of remaining capacity compared with 24 W for FCFS.

Conclusion

This simulation demonstrates how execution level power predictions can be incorporated into a simple infrastructure scheduling decision.

Across the tested hypothetical power budgets, the power aware strategy avoided budget violations under tighter constraints and reduced actual scheduled power when the same number of workloads could be accommodated. At the 10 kW budget, for example, the strategy scheduled the same four workloads while leaving approximately 1.91 kW of additional power headroom compared with the FCFS baseline.

These findings demonstrate the potential of the ML model to support power aware capacity planning and workload scheduling. The results are simulation-based and should not be interpreted as measured production savings or an optimal scheduling policy.